

CSA1708-ARTIFICIAL INTELLIGENCE

LAB EXPERIMENTS

Program 1 : Python Program to Solve 8. Puzzle Problem

Aim

To develop a Python program that solves the 8-Puzzle Problem using a search algorithm (such as Breadth-First Search - BFS) to find the shortest sequence of moves from the initial state to the goal state.

Algorithm

1. Represent the puzzle as a 3×3 matrix.
2. Define the goal state.
3. Locate the empty tile (0).
4. Generate all possible moves (up, down, left, right).
5. Use Breadth-First Search (BFS) to explore possible states.
6. Keep track of visited states to avoid repeated exploration.
7. If the goal state is reached, display the solution path.
8. If all possibilities are exhausted, report that no solution exists.

```

S PUZZLE.py - C:/Users/jasmi/OneDrive/Pictures/Documents/S PUZZLE.py (3.14.4)
File Edit Format Run Options Window Help
from collections import deque

GOAL = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 0)
)

def get_neighbors(state):
    neighbors = []

    # Find the blank tile (0)
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dx, dy in directions:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]

            new_state[x][y], new_state[nx][ny] = (
                new_state[nx][ny],
                new_state[x][y],
            )

            neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors

def bfs(start):
    queue = deque([(start, [])])
    visited = set()

    while queue:
        state, path = queue.popleft()

        if state == GOAL:
            return path + [state]

        if state in visited:
            continue

```

```

IDLE Shell 3.14.4
File Edit Shell Debug Options Window Help
Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/S PUZZLE.py =====
Solution Found!
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
>>> |

```

RESULT

Thus, the 8-Puzzle Problem solved successfully using Python.

Program 2: Python Program to Solve 8 Queen Problem

Aim

To develop a Python program to solve the 8-Queen Problem using the Backtracking Algorithm, such that no two queens attack each other on an 8×8 chessboard.

Algorithm

1. Create an empty 8×8 chessboard.
2. Start placing queens row by row.
3. Before placing a queen, check whether the position is safe:
 - No queen exists in the same column.
 - No queen exists in the left diagonal.
 - No queen exists in the right diagonal.
4. If the position is safe, place the queen.
5. Recursively place queens in the next row.
6. If no safe position is found, backtrack by removing the previously placed queen.
7. Continue until all 8 queens are successfully placed.
8. Display the final chessboard configuration.

N-Queens Problem using Backtracking

N = 8

Print the board

```
def print_board(board):  
    for row in board:  
        for cell in row:  
            if cell:  
                print("Q", end=" ")  
            else:  
                print(".", end=" ")  
        print()
```

Check if placing queen is safe

```
def is_safe(board, row, col):  
  
    # Check column  
    for i in range(row):  
        if board[i][col]:  
            return False  
  
    # Check upper-left diagonal  
    i, j = row - 1, col - 1  
    while i >= 0 and j >= 0:  
        if board[i][j]:  
            return False  
        i -= 1  
        j -= 1  
  
    # Check upper-right diagonal  
    i, j = row - 1, col + 1  
    while i >= 0 and j < N:  
        if board[i][j]:  
            return False  
        i -= 1  
        j += 1  
  
    return True
```

Backtracking function

```
def solve(board, row):  
    if row == N:  
        return True  
  
    for col in range(N):  
        if is_safe(board, row, col):
```

```
Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/S PUZZLE.py =====
Solution Found!

(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
>>>
===== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/8 QUEEN.py =====
Solution Found:

Q . . . . .
. . . . Q . .
. . . . . Q
. . . . Q . .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . .
>>>
```

RESULT:

Thus, the 8-Queen Problem was solved successfully using Python.

Program 3: Water Jug Problem using Python

Aim

To solve the Water Jug Problem using state-space search.

Algorithm

1. Start.
2. Initialize the capacities of Jug 1, Jug 2, and the target amount.
3. Create the initial state (0, 0).
4. Create an empty queue and insert the initial state.
5. Create an empty set to store visited states.
6. While the queue is not empty:
 - Remove the front state from the queue.
 - If the state is already visited, continue to the next state.
 - Mark the current state as visited.
 - If either jug contains the target amount, print the solution path and stop.
 - Generate the following successor states:
 - Fill Jug 1.
 - Fill Jug 2.
 - Empty Jug 1.
 - Empty Jug 2.
 - Pour water from Jug 1 to Jug 2.
 - Pour water from Jug 2 to Jug 1.

- Add all unvisited successor states to the queue.
7. If the queue becomes empty, print "**No Solution**".
 8. Stop.

water.py - C:/Users/jasmi/OneDrive/Desktop/water.py (3.4.3)

File Edit Format Run Options Window Help

```
from collections import deque

def water_jug(jug1, jug2, target):
    visited = set()
    queue = deque([(0, 0)])

    while queue:
        x, y = queue.popleft()

        if (x, y) in visited:
            continue
        visited.add((x, y))

        print((x, y))

        if x == target or y == target:
            print("Target reached")
            return

        next_states = [
            (jug1, y),
            (x, jug2),
            (0, y),
            (x, 0),
            (max(0, x-(jug2-y)), min(jug2, y+x)),
            (min(jug1, x+y), max(0, y-(jug1-x)))
        ]

        for state in next_states:
            if state not in visited:
                queue.append(state)

water_jug(4, 3, 2)
```

```
Python 3.4.3 Shell
File Edit Shell Debug Options Window Help
Python 3.4.3 (v3.4.3:9b73flc3e601, Feb 24 2015, 22:44:40) [MSC v.1600 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
(0, 0)
(4, 0)
(0, 3)
(4, 3)
(1, 3)
(3, 0)
(1, 0)
(3, 3)
(0, 1)
(4, 2)
Target reached
>>>
```

Result :

Thus, the Water Jug Problem was solved successfully using Python.

Program 4: Missionaries and Cannibals Problem

Aim:

To solve the Missionaries and Cannibals problem using State Space Search in Python.

Algorithm:

1. Start with the initial state (3, 3, Left Bank).
2. Create a queue and insert the initial state.
3. Create a set to store visited states.
4. Remove the front state from the queue.
5. If it is the goal state (0, 0, Right Bank), print the solution path and stop.
6. Generate all possible boat moves:
 - Two missionaries
 - Two cannibals
 - One missionary
 - One cannibal
 - One missionary and one cannibal
7. For each generated state:
 - Check whether the state is valid (missionaries are never outnumbered by cannibals on either bank).
 - If valid and not visited, add it to the queue.
8. Repeat Steps 4–7 until the goal state is reached.
9. Display the sequence of states from the initial state to the goal state.

```
    if m_left < 0 or c_left < 0 or m_left > 3 or c_left > 3:
        return False

    if (m_left > 0 and m_left < c_left):
        return False

    if (m_right > 0 and m_right < c_right):
        return False

    return True

def missionaries_cannibals():
    start = (3, 3, 1)
    goal = (0, 0, 0)

    queue = deque([(start, [])])
    visited = set()

    while queue:
        (m, c, boat), path = queue.popleft()

        if (m, c, boat) in visited:
            continue

        visited.add((m, c, boat))
        path = path + [(m, c, boat)]

        if (m, c, boat) == goal:
            print("Solution Path:")
            for state in path:
                print(state)
            return

        moves = [(2, 0), (0, 2), (1, 0), (0, 1), (1, 1)]

        for dm, dc in moves:
            if boat == 1:
                new = (m-dm, c-dc, 0)
            else:
                new = (m+dm, c+dc, 1)

            if is_valid(new[0], new[1]):
                queue.append((new, path))

missionaries_cannibals()
```

```
Python 3.4.3 Shell
File Edit Shell Debug Options Window Help
Python 3.4.3 (v3.4.3:9b73f1c3e601, Feb 24 2015, 22:44:40) [MSC v.1600 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
(0, 0)
(4, 0)
(0, 3)
(4, 3)
(1, 3)
(3, 0)
(1, 0)
(3, 3)
(0, 1)
(4, 2)
Target reached
>>> ===== RESTART =====
>>>
Solution Path:
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
>>> |
```

Result

Thus, the Missionaries and Cannibals Problem was successfully solved using State Space Search (Breadth-First Search) in Python.

Program 5: Vacuum Cleaner Problem using Python

Aim

To implement the Vacuum Cleaner Problem using Python and clean all dirty rooms.

Algorithm

1. Start.
2. Initialize the environment with rooms and their states (Clean or Dirty).
3. Set the vacuum cleaner's initial position.
4. Check the current room.
5. If the current room is dirty, clean it.
6. Move the vacuum cleaner to the next room.
7. Repeat Steps 4–6 until all rooms have been visited.
8. Display the final state of all rooms.
9. Stop.

```

vacuum cleaner.py - C:/Users/jasmi/OneDrive/Desktop/vacuum cleaner.py (
File Edit Format Run Options Window Help
def vacuum_cleaner(rooms):
    for room in rooms:
        print(f"Checking Room {room}")

        if rooms[room] == "Dirty":
            print(f"Room {room} is Dirty.")
            print("Cleaning...")
            rooms[room] = "Clean"
        else:
            print(f"Room {room} is already Clean.")

        print()

    print("Final State of Rooms:")
    for room in rooms:
        print(f"Room {room}: {rooms[room]}")

# Initial State
rooms = {
    "A": "Dirty",
    "B": "Clean",
    "C": "Dirty",
    "D": "Dirty"
}

vacuum_cleaner(rooms)

```

```

IDLE Shell 3.14.4
File Edit Shell Debug Options Window Help
Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10)
Enter "help" below or click "Help" above for more info:
>>>
===== RESTART: C:/Users/jasmi/OneDrive/Desktop/vacuum cleaner.py
Checking Room A
Room A is Dirty.
Cleaning...

Checking Room B
Room B is already Clean.

Checking Room C
Room C is Dirty.
Cleaning...

Checking Room D
Room D is Dirty.
Cleaning...

Final State of Rooms:
Room A: Clean
Room B: Clean
Room C: Clean
Room D: Clean
>>>

```

Result

Thus, the Vacuum Cleaner Problem was successfully using the python.

Program 6: Breadth-First Search (BFS)

AIM

To implement Breadth-First Search (BFS) in Python to traverse all the vertices of a graph level by level using a queue.

ALGORITHM

1. Start with a graph and select the starting vertex.
2. Create an empty visited set to store visited vertices.
3. Create a queue and insert the starting vertex.
4. Mark the starting vertex as visited.
5. Repeat until the queue is empty:
 - Remove the front vertex from the queue.
 - Display the vertex.
 - Check all its adjacent vertices.
 - If an adjacent vertex is not visited, mark it visited and add it to the queue.
6. Stop when the queue becomes empty.


```
Breadth-First Search (BFS).py - C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/Br...
File Edit Format Run Options Window Help

from collections import deque

# Function to perform BFS
def bfs(graph, start):
    visited = set()          # To keep track of visited nodes
    queue = deque([start])   # Initialize queue with the start node
    visited.add(start)

    print("BFS Traversal:")

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        # Visit all unvisited neighbors
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# Example graph represented as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

# Starting node
start_node = 'A'

# Call BFS
bfs(graph, start_node)
```

```
IDLE Shell 3.14.4
File Edit Shell Debug Options Window Help

Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/Breadth-First Search (BFS).py
      BFS Traversal:
      A B C D E F
>>> |

Ln: 7 Col: 0
```

RESULT

The BFS program was successfully implemented in Python. The graph was traversed level by level.

Program 7: Depth-First Search (DFS)

AIM

To implement Depth-First Search (DFS) in Python to traverse all the vertices of a graph by exploring as far as possible along each branch before backtracking.

ALGORITHM

1. Start with a graph and select the starting vertex.
2. Create an empty visited set.
3. Mark the starting vertex as visited.
4. Display the starting vertex.
5. Visit each adjacent vertex that has not been visited.
6. Recursively apply DFS to the unvisited adjacent vertex.
7. Continue until all reachable vertices are visited.
8. Stop the traversal.

```
DFS.py - C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/DFS.py (3.14.4)
File Edit Format Run Options Window Help

# Function to perform DFS
def dfs(graph, node, visited):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbor in graph[node]:
            dfs(graph, neighbor, visited)

# Graph represented as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

visited = set()

print("DFS Traversal:")
dfs(graph, 'A', visited)
```

```
IDLE Shell 3.14.4
File Edit Shell Debug Options Window Help
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/DFS.py =====
DFS Traversal:
A B D E F C
>>>
```

RESULT

The Depth-First Search (DFS) algorithm was successfully implemented in Python, and the graph was traversed by exploring each branch deeply before backtracking.

Program 8: Travelling Salesman Problem (TSP)

AIM

To implement the Travelling Salesman Problem (TSP) using the Brute Force / Exhaustive Search approach in Python and find the minimum-cost route that visits every city exactly once and returns to the starting city.

ALGORITHM

1. Start with a set of cities and their distance/cost matrix.
2. Select the first city as the starting city.
3. Generate all possible permutations of the remaining cities.
4. For each permutation:
 - Calculate the total distance of the tour.
 - Add the distance from the last city back to the starting city.
5. Compare the total distances of all possible tours.
6. Select the tour with the minimum distance.
7. Display the shortest route and its minimum cost.
8. Stop.

```
TRAVELLING SALESMAN.py - C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/TRA...
File Edit Format Run Options Window Help
from itertools import permutations

# Distance matrix
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

cities = [0, 1, 2, 3]

min_distance = float('inf')
best_route = None

# Generate all permutations of cities except starting city
for route in permutations(cities[1:]):
    current_route = (0,) + route + (0,)
    distance = 0

    # Calculate total distance
    for i in range(len(current_route) - 1):
        distance += graph[current_route[i]][current_route[i + 1]]

    # Find minimum distance
    if distance < min_distance:
        min_distance = distance
        best_route = current_route

print("Best Route:", best_route)
print("Minimum Distance:", min_distance)
```

```
File Edit Shell Debug Options Window Help
Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/TRAVELLING SALESMAN.py
Best Route: (0, 1, 3, 2, 0)
Minimum Distance: 80
>>>
```

RESULT

The Travelling Salesman Problem was successfully implemented using the Brute Force approach, and the minimum-cost tour was found as $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$, with a total distance of 80.

Program 9: A*Algorithm

AIM

To implement the Travelling Salesman Problem (TSP) using the Brute Force / Exhaustive Search approach in Python and find the minimum-cost route that visits every city exactly once and returns to the starting city.

ALGORITHM

1. Start with a set of cities and their distance/cost matrix.
2. Select the first city as the starting city.
3. Generate all possible permutations of the remaining cities.
4. For each permutation:
 - Calculate the total distance of the tour.
 - Add the distance from the last city back to the starting city.
5. Compare the total distances of all possible tours.
6. Select the tour with the minimum distance.
7. Display the shortest route and its minimum cost.
8. Stop.

9.py - C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/9.py (3.14.4)

File Edit Format Run Options Window Help

```
import heapq

def a_star(graph, heuristic, start, goal):
    # Priority queue
    open_list = []
    heapq.heappush(open_list, (heuristic[start], 0, start, [start]))

    visited = set()

    while open_list:
        f, g, current, path = heapq.heappop(open_list)

        if current == goal:
            return path, g

        if current in visited:
            continue

        visited.add(current)

        for neighbor, cost in graph[current]:
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbor]

                heapq.heappush(
                    open_list,
                    (new_f, new_g, neighbor, path + [neighbor])
                )

    return None, float('inf')
```

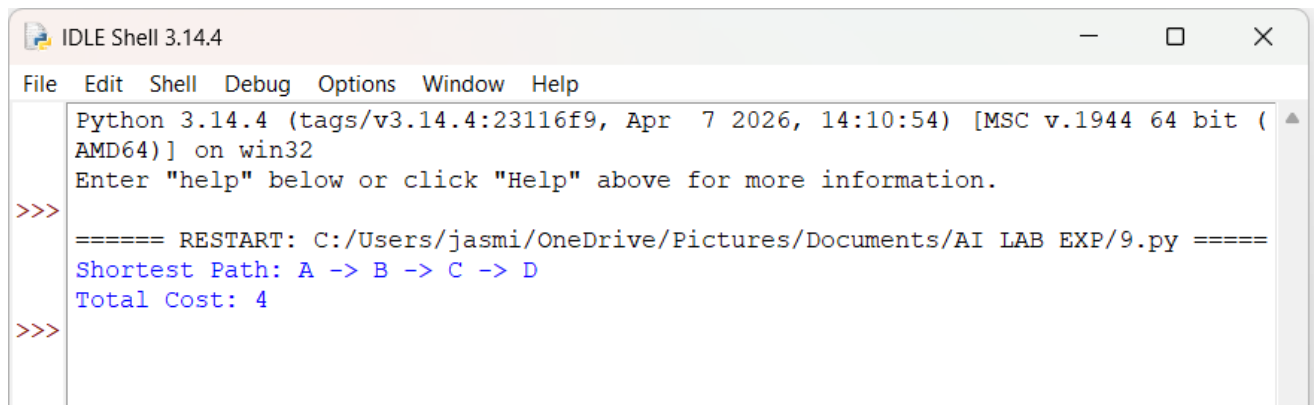
```
# Graph
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('A', 1), ('C', 2), ('D', 5)],
    'C': [('A', 4), ('B', 2), ('D', 1)],
    'D': [('B', 5), ('C', 1)]
}

# Heuristic values
heuristic = {
    'A': 4,
    'B': 2,
    'C': 1,
    'D': 0
}

start = 'A'
goal = 'D'

path, cost = a_star(graph, heuristic, start, goal)

print("Shortest Path:", " -> ".join(path))
print("Total Cost:", cost)
```



```
IDLE Shell 3.14.4
File Edit Shell Debug Options Window Help
Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/9.py =====
Shortest Path: A -> B -> C -> D
Total Cost: 4
>>>
```

RESULT

The A* Search Algorithm was successfully implemented in Python. The shortest path from A to D was found as:

$A \rightarrow B \rightarrow C \rightarrow D$

with a total cost of 4.

Program 10: Map Colouring Problem using Constraint Satisfaction Problem (CSP)

AIM

To implement the Map Colouring Problem using Constraint Satisfaction Problem (CSP) in Python, where adjacent regions are assigned different colors.

ALGORITHM

1. Define the regions of the map as variables.
2. Define the available colors as the domain for each region.
3. Define the constraints such that adjacent regions cannot have the same color.
4. Select an uncolored region.
5. Assign a color from its domain.
6. Check whether the assignment satisfies all constraints.
7. If valid, continue with the next region.
8. If invalid, try another color.
9. If no color is possible, backtrack to the previous region.
10. Continue until all regions are colored.
11. Display the valid map coloring.

untitled (3.14.4)

File Edit Format Run Options Window Help

```
# Map Coloring using CSP

# Variables
regions = ['A', 'B', 'C', 'D']

# Available colors
colors = ['Red', 'Green', 'Blue']

# Constraints: adjacent regions
neighbors = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

# Store color assignments
assignment = {}

# Check whether a color is valid
def is_valid(region, color):
    for neighbor in neighbors[region]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False
    return True

# Backtracking CSP
def map_coloring():
    if len(assignment) == len(regions):
        return True

    # Select an uncolored region
    for region in regions:
        if region not in assignment:
            break

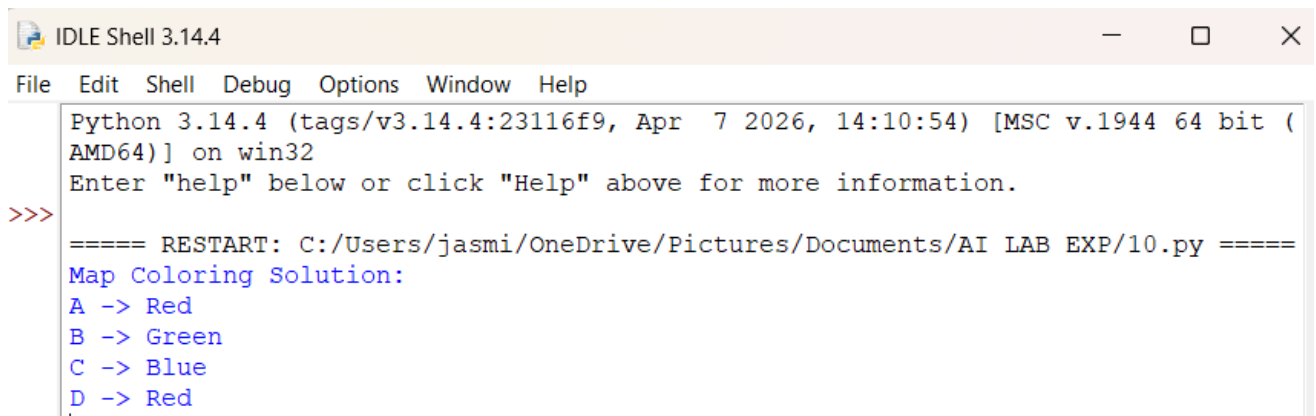
    # Try each color
    for color in colors:
        if is_valid(region, color):
            assignment[region] = color

            if map_coloring():
                return True

            # Backtrack
            del assignment[region]

    return False

# Solve the problem
if map_coloring():
    print("Map Coloring Solution:")
    for region in regions:
        print(region, "->", assignment[region])
else:
    print("No solution exists.")
```



```
IDLE Shell 3.14.4
File Edit Shell Debug Options Window Help
Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/10.py =====
Map Coloring Solution:
A -> Red
B -> Green
C -> Blue
D -> Red
```

RESULT

The Map Coloring Problem was successfully implemented using the Constraint Satisfaction Problem (CSP) approach. All adjacent regions were assigned different colors while satisfying the given constraints.

Program 11: Tic-Tac-Toe game

AIM

To implement a Tic-Tac-Toe game in Python where two players can play against each other by placing X and O on a 3×3 game board.

ALGORITHM

1. Create a 3×3 board and initialize all positions as empty.
 2. Display the board.
 3. Set the first player as X.
 4. Ask the current player to enter a position from 1 to 9.
 5. Check whether the selected position is empty.
 6. Place the player's symbol in that position.
 7. Check whether the player has three symbols in a row, column, or diagonal.
 8. If a player wins, display the winner and stop the game.
 9. If all positions are filled and nobody wins, declare the game as a draw.
- Otherwise, switch to the other player and repeat the process.

```

# Tic-Tac-Toe Game

board = [' ' for _ in range(9)]

def display_board():
    print()
    print(board[0] + " | " + board[1] + " | " + board[2])
    print("---+---+---")
    print(board[3] + " | " + board[4] + " | " + board[5])
    print("---+---+---")
    print(board[6] + " | " + board[7] + " | " + board[8])
    print()

def check_winner(player):
    winning_positions = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]

    for a, b, c in winning_positions:
        if board[a] == board[b] == board[c] == player:
            return True

    return False

def play_game():
    current_player = 'X'

    for turn in range(9):
        display_board()

        print("Player", current_player)
        position = int(input("Enter position (1-9): ")) - 1

        print("Player", current_player)
        position = int(input("Enter position (1-9): ")) - 1

        if position < 0 or position > 8 or board[position] != ' ':
            print("Invalid position! Try again.")
            continue

        board[position] = current_player

        if check_winner(current_player):
            display_board()
            print("Player", current_player, "wins!")
            return

        current_player = 'O' if current_player == 'X' else 'X'

    display_board()
    print("Game Draw!")

play_game()

```

```
IDLE Shell 3.14.4
File Edit Shell Debug Options Window Help

Player X
Enter position (1-9): 4

  |  |
--+--+
X |  |
--+--+
  |  |

Player O
Enter position (1-9): 9

  |  |
--+--+
X |  |
--+--+
  |  | O

Player X
Enter position (1-9): 7

  |  |
--+--+
X |  |
--+--+
X |  | O

Player O
Enter position (1-9): 5

  |  |
--+--+
X | O |
--+--+
X |  | O

Player X
Enter position (1-9): 1

X |  |
--+--+
X | O |
--+--+
X |  | O

Player X wins!
>>> |
```

RESULT

The Tic-Tac-Toe game was successfully implemented in Python. The program allows two players to play alternately, checks for winning combinations, and identifies a winner or draw.

Program 12: Minimax Algorithm for Gaming

AIM

To implement the Minimax Algorithm for Gaming in Python and use it to determine the best possible move for a player in a Tic-Tac-Toe game.

ALGORITHM

1. Create a 3×3 Tic-Tac-Toe board.
2. Define the possible winning combinations.
3. Check whether the current board is a win, loss, or draw.
4. Generate all possible moves.
5. For each possible move:
 - Make the move.
 - Recursively call the Minimax function.
 - Undo the move.
6. For the MAX player, select the move with the highest score.
7. For the MIN player, select the move with the lowest score.
8. Continue until a terminal state is reached.
9. Return the best move and its score.
10. Display the best move selected by the Minimax algorithm.

```
# Minimax Algorithm for Tic-Tac-Toe

board = [' ' for _ in range(9)]

def display_board():
    print()
    print(board[0], '|', board[1], '|', board[2])
    print('---+---+---')
    print(board[3], '|', board[4], '|', board[5])
    print('---+---+---')
    print(board[6], '|', board[7], '|', board[8])
    print()

def check_winner(player):
    winning_positions = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8),
        (0, 3, 6), (1, 4, 7), (2, 5, 8),
        (0, 4, 8), (2, 4, 6)
    ]

    for a, b, c in winning_positions:
        if board[a] == board[b] == board[c] == player:
            return True

    return False

def minimax(is_maximizing):
    if check_winner('O'):
        return 1

    if check_winner('X'):
        return -1

    if ' ' not in board:
        return 0

    if is_maximizing:
        best_score = -float('inf')
```



```

        best_score = max(best_score, score)

    return best_score

else:
    best_score = float('inf')

    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            score = minimax(True)
            board[i] = ' '
            best_score = min(best_score, score)

    return best_score

def best_move():
    best_score = -float('inf')
    move = -1

    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            score = minimax(False)
            board[i] = ' '

            if score > best_score:
                best_score = score
                move = i

    return move

# Example game
board[0] = 'X'
board[4] = 'O'
board[1] = 'X'

display_board()

move = best_move()
board[move] = 'O'

print("Best move selected by Minimax:", move + 1)
display_board()

```

Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)] on win32

Enter "help" below or click "Help" above for more information.

>>>

===== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/12.py =====

```

X | X |
--+---+--
| O |
--+---+--
|   |

```

Best move selected by Minimax: 3

```

X | X | O
--+---+--
| O |
--+---+--
|   |

```

RESULT

The Minimax Algorithm was successfully implemented for gaming. The algorithm evaluates all possible future moves and selects the optimal move for the computer player in the Tic-Tac-Toe game.

Program 13: Alpha-Beta Pruning Algorithm for Gaming

AIM

To implement the Alpha-Beta Pruning Algorithm for Gaming in Python and improve the Minimax algorithm by eliminating branches that do not need to be evaluated.

ALGORITHM

1. Initialize the game board.
2. Define the winning conditions.
3. Start the Alpha-Beta algorithm with:
 - $\alpha = -\infty$
 - $\beta = +\infty$
4. Generate all possible moves.
5. For the MAX player, select the move with the highest score.
6. Update alpha with the maximum value found.
7. For the MIN player, select the move with the lowest score.
8. Update beta with the minimum value found.
9. If $\alpha \geq \beta$, stop evaluating the remaining branches (pruning).
10. Continue until a winning, losing, or draw state is reached.
11. Select and display the best move.

```
# Alpha-Beta Pruning Algorithm for Tic-Tac-Toe

board = [' ' for _ in range(9)]

def display_board():
    print()
    print(board[0], '|', board[1], '|', board[2])
    print('---+---+---')
    print(board[3], '|', board[4], '|', board[5])
    print('---+---+---')
    print(board[6], '|', board[7], '|', board[8])
    print()

def check_winner(player):
    winning_positions = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8),
        (0, 3, 6), (1, 4, 7), (2, 5, 8),
        (0, 4, 8), (2, 4, 6)
    ]

    for a, b, c in winning_positions:
        if board[a] == board[b] == board[c] == player:
            return True

    return False

def alpha_beta(is_maximizing, alpha, beta):

    # Computer wins
    if check_winner('O'):
        return 1

    # Human wins
    if check_winner('X'):
        return -1

    # Draw
    if ' ' not in board:
        return 0

    if is_maximizing:
        best_score = -float('inf')

        for i in range(9):
```

```

        best_score = min(best_score, score)
        beta = min(beta, best_score)

        # Alpha-Beta pruning
        if beta <= alpha:
            break

    return best_score

def best_move():
    best_score = -float('inf')
    move = -1

    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'

            score = alpha_beta(
                False,
                -float('inf'),
                float('inf')
            )

            board[i] = ' '

            if score > best_score:
                best_score = score
                move = i

    return move

# Example board
board[0] = 'X'
board[4] = 'O'
board[1] = 'X'

display_board()

move = best_move()
board[move] = 'O'

print("Best move selected:", move + 1)

display_board()

```

```
IDLE Shell 3.14.4
File Edit Shell Debug Options Window Help
Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/13.py =====

X | X |
--+---+--
  | O |
--+---+--
  |   |

Best move selected: 3

X | X | O
--+---+--
  | O |
--+---+--
  |   |
```

RESULT

The Alpha-Beta Pruning Algorithm was successfully implemented for gaming. It finds the optimal move like Minimax while pruning unnecessary branches, reducing the number of game states that need to be evaluated.

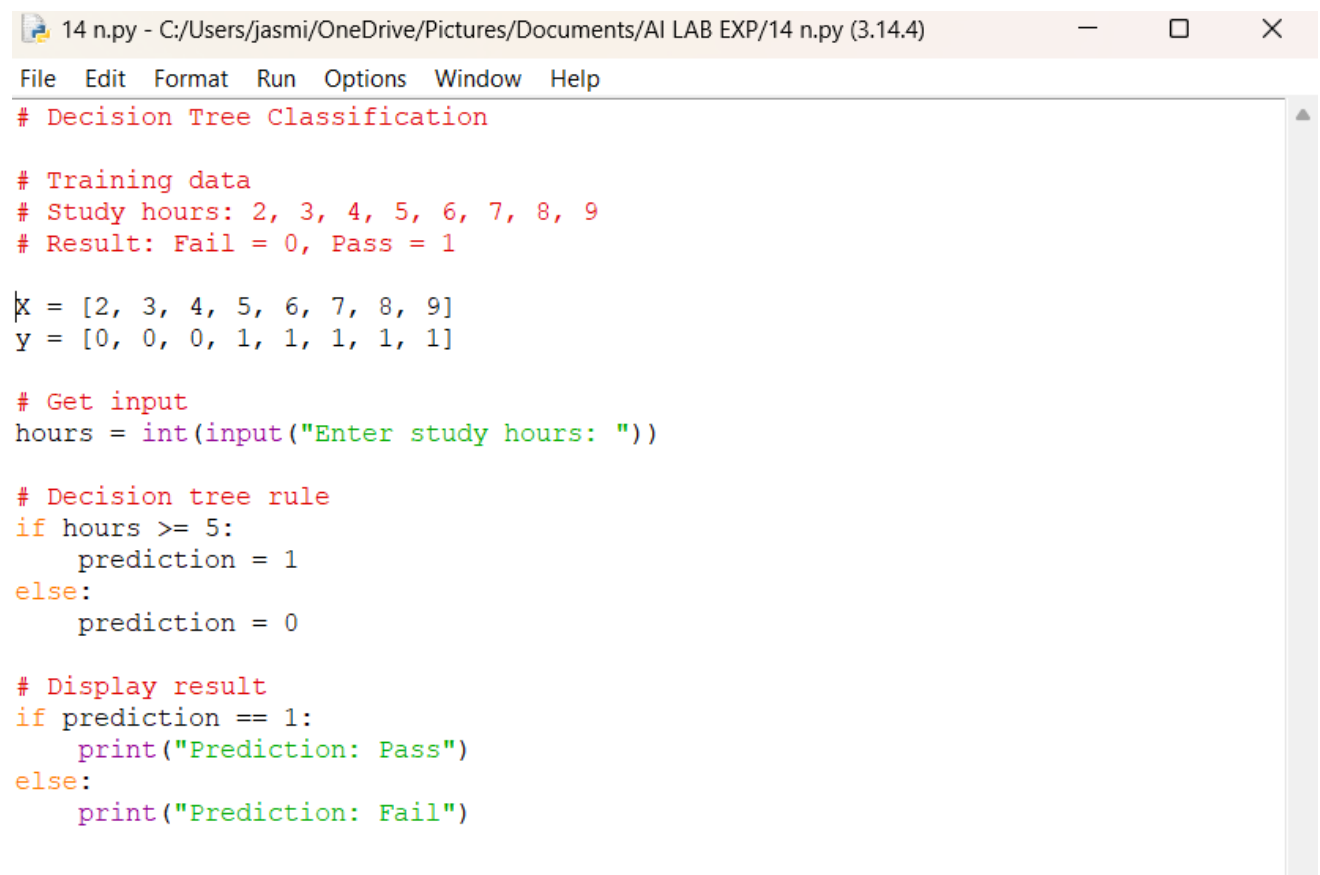
Program 14: Decision Tree Classification algorithm

AIM

To implement a Decision Tree Classification algorithm in Python and classify data based on different feature conditions.

ALGORITHM

1. Import the required libraries.
2. Create a dataset containing features and target classes.
3. Separate the input features (X) and target variable (y).
4. Create a Decision Tree Classifier.
5. Train the model using the dataset.
6. Give a new input sample to the trained model.
7. Predict the class of the new sample.
8. Display the predicted result.

A screenshot of a Python IDE window titled "14 n.py - C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/14 n.py (3.14.4)". The window contains a Python script for a Decision Tree Classification algorithm. The script defines training data (X and y), gets user input for study hours, and implements a simple decision rule to predict Pass or Fail based on whether study hours are greater than or equal to 5. The code is color-coded: comments are red, keywords are blue, and strings are green.

```
14 n.py - C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/14 n.py (3.14.4)
File Edit Format Run Options Window Help
# Decision Tree Classification

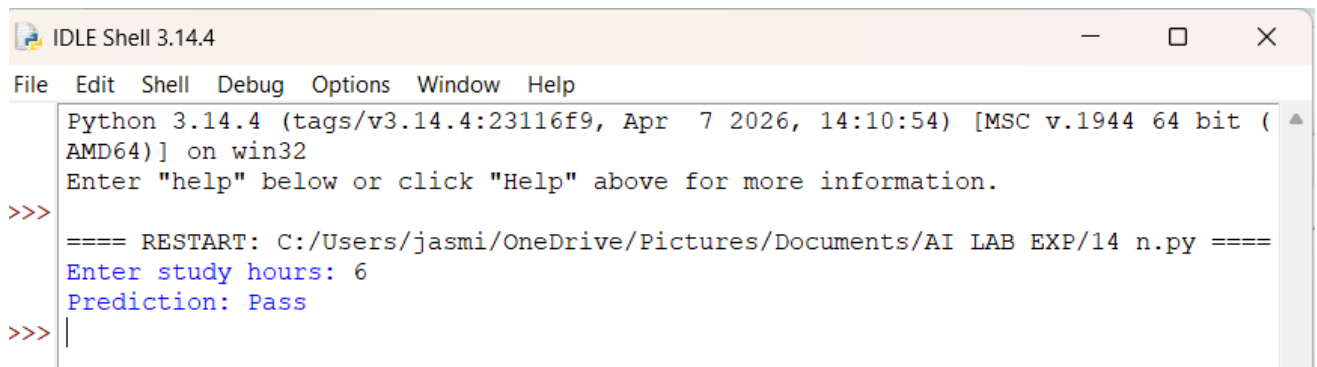
# Training data
# Study hours: 2, 3, 4, 5, 6, 7, 8, 9
# Result: Fail = 0, Pass = 1

X = [2, 3, 4, 5, 6, 7, 8, 9]
y = [0, 0, 0, 1, 1, 1, 1, 1]

# Get input
hours = int(input("Enter study hours: "))

# Decision tree rule
if hours >= 5:
    prediction = 1
else:
    prediction = 0

# Display result
if prediction == 1:
    print("Prediction: Pass")
else:
    print("Prediction: Fail")
```

A screenshot of the IDLE Shell 3.14.4 window. The title bar shows 'IDLE Shell 3.14.4' and standard window controls. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The shell area displays the following text: 'Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)] on win32', 'Enter "help" below or click "Help" above for more information.', a red prompt '>>>', a blue restart message '==== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/14 n.py ====', a blue prompt 'Enter study hours: 6', a blue output 'Prediction: Pass', and another red prompt '>>>' followed by a vertical cursor.

```
Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/14 n.py ====
Enter study hours: 6
Prediction: Pass
>>> |
```

RESULT

The Decision Tree Classification algorithm was successfully implemented in Python. The trained decision tree classified the given student data and predicted Pass based on study hours and attendance.

Program 15: Feed Forward Neural Network (FFNN)

AIM

To implement a Feed Forward Neural Network (FFNN) in Python and use it to classify input data.

ALGORITHM

1. Import the required libraries.
2. Create the input dataset and corresponding output labels.
3. Create a Feed Forward Neural Network.
4. Define the input layer, hidden layer, and output layer.
5. Train the neural network using the training data.
6. Provide a new input value to the trained network.
7. Predict the output.
8. Display the predicted result.

```
15n.py - C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/15n.py (3.14.4)
File Edit Format Run Options Window Help
import math

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + math.exp(-x))

# Input values
x1 = float(input("Enter first input (0 or 1): "))
x2 = float(input("Enter second input (0 or 1): "))

# Weights and bias
w1 = 1
w2 = 1
bias = -1.5

# Feed Forward calculation
weighted_sum = (x1 * w1) + (x2 * w2) + bias

# Activation
output = sigmoid(weighted_sum)

print("Neural Network Output:", round(output, 2))

# Classification
if output >= 0.5:
    print("Prediction: 1")
else:
    print("Prediction: 0")
```

```
IDLE Shell 3.14.4
File Edit Shell Debug Options Window Help
Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/15.py =====
Enter first input (0 or 1):
===== RESTART: C:/Users/jasmi/OneDrive/Pictures/Documents/AI LAB EXP/15n.py =====
Enter first input (0 or 1): 1
Enter second input (0 or 1): 0
Neural Network Output: 0.38
Prediction: 0
```

RESULT

The Feed Forward Neural Network was successfully implemented in Python and correctly predicted the output for the given input values.

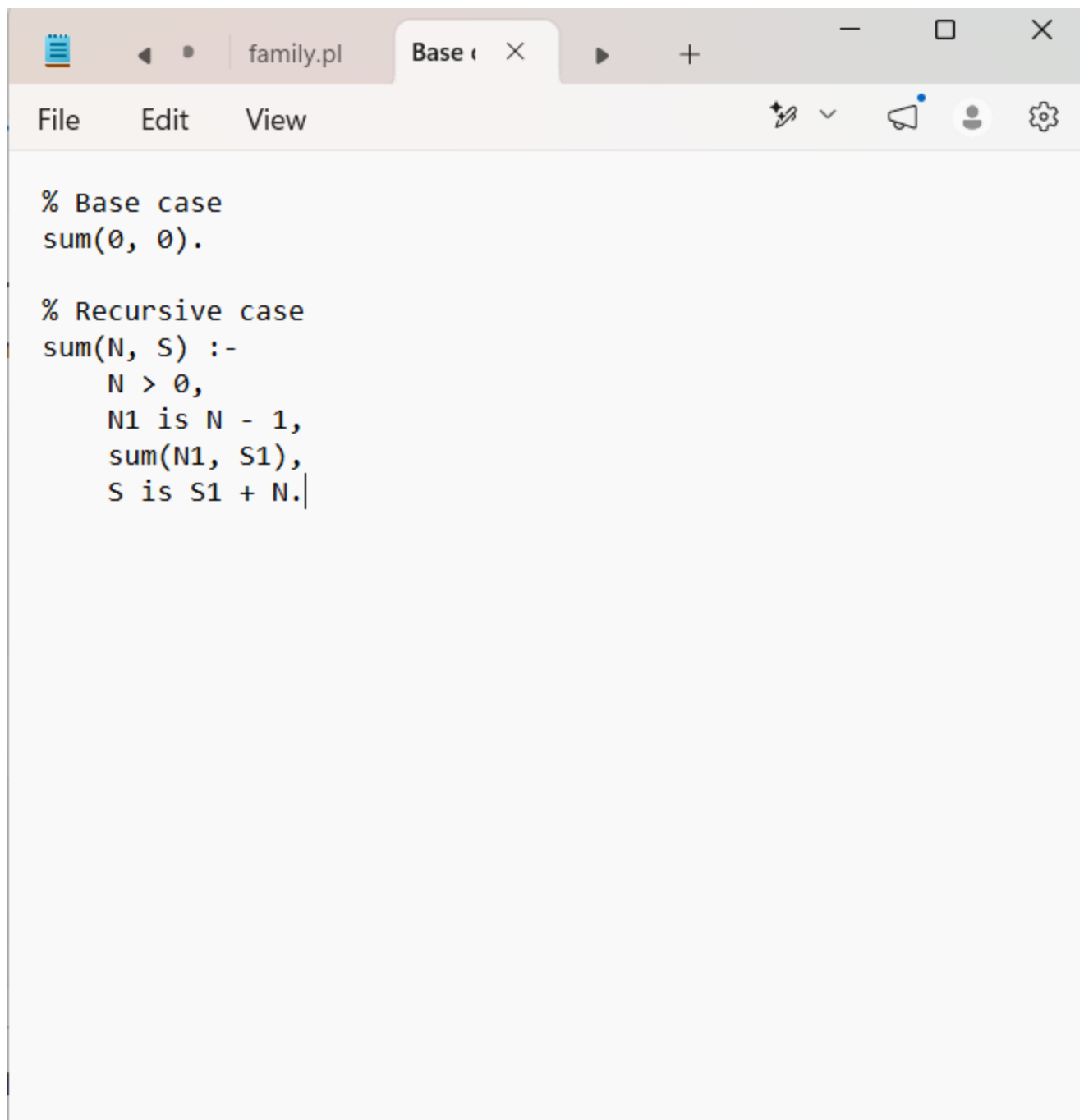
Program 17: Find the sum of integers from 1 to N.

AIM

To write a Prolog program to find the sum of integers from 1 to N.

ALGORITHM

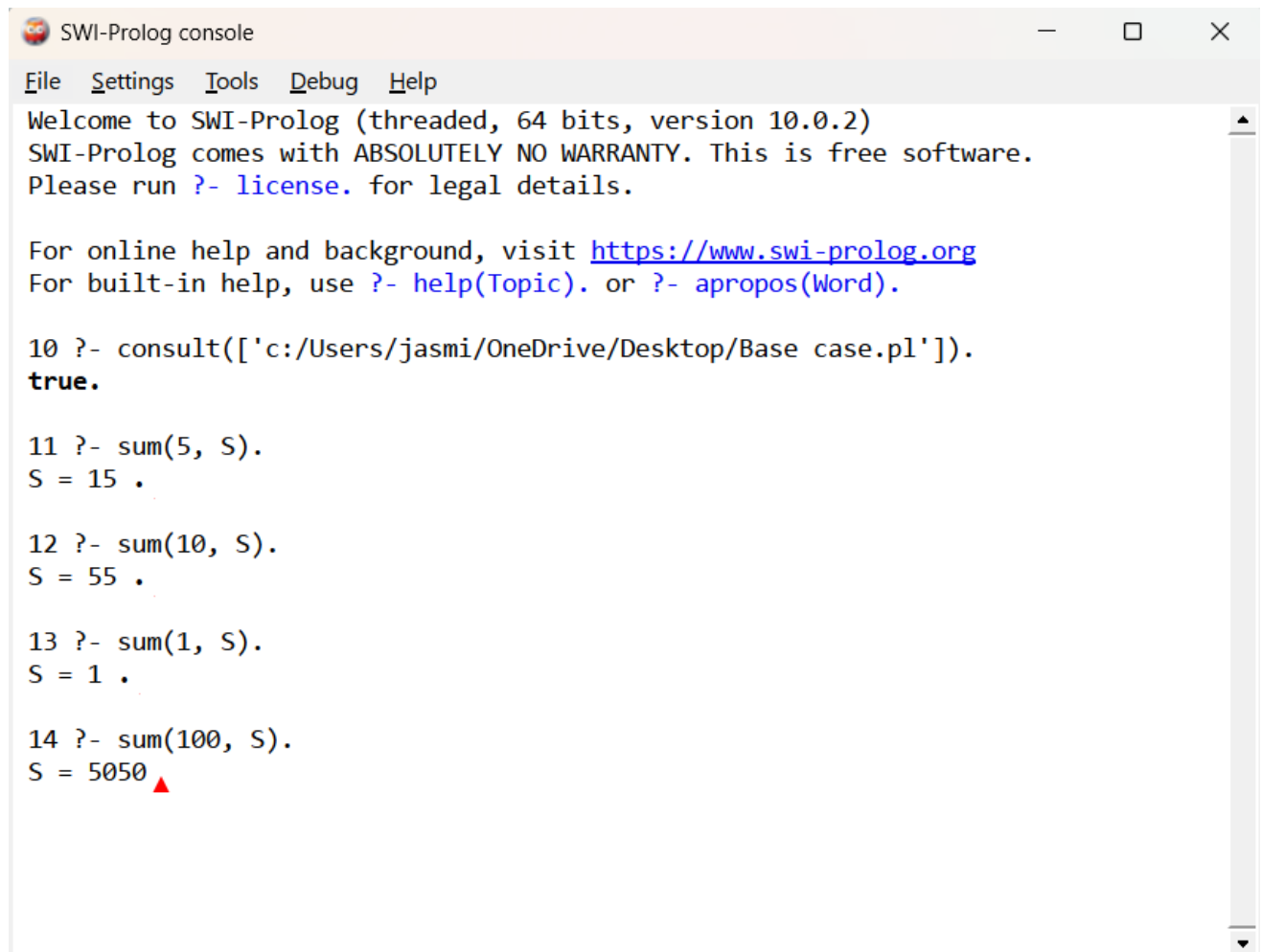
1. Start.
2. Read the value of N.
3. Initialize sum = 0.
4. Repeat from 1 to N.
5. Add each integer to sum.
6. Display the final sum.
7. Stop.



The image shows a screenshot of a Prolog IDE window. The window has a title bar with a file icon, a back button, a file name 'family.pl', a tab name 'Base c', and window control buttons (minimize, maximize, close). Below the title bar is a menu bar with 'File', 'Edit', and 'View'. To the right of the menu bar are several icons: a star, a dropdown arrow, a megaphone, a user profile, and a settings gear. The main area of the window contains the following Prolog code:

```
% Base case
sum(0, 0).

% Recursive case
sum(N, S) :-
    N > 0,
    N1 is N - 1,
    sum(N1, S1),
    S is S1 + N.
```



The image shows a screenshot of the SWI-Prolog console window. The window has a title bar with the text "SWI-Prolog console" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with options: File, Settings, Tools, Debug, and Help. The main text area contains the following content:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

10 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/Base case.pl']).
true.

11 ?- sum(5, S).
S = 15 .

12 ?- sum(10, S).
S = 55 .

13 ?- sum(1, S).
S = 1 .

14 ?- sum(100, S).
S = 5050 ▲
```

The output shows that the Prolog program successfully calculated the sum of integers from 1 to N for N values of 5, 10, 1, and 100. The results are S = 15, S = 55, S = 1, and S = 5050 respectively. A red triangle cursor is visible at the end of the last line.

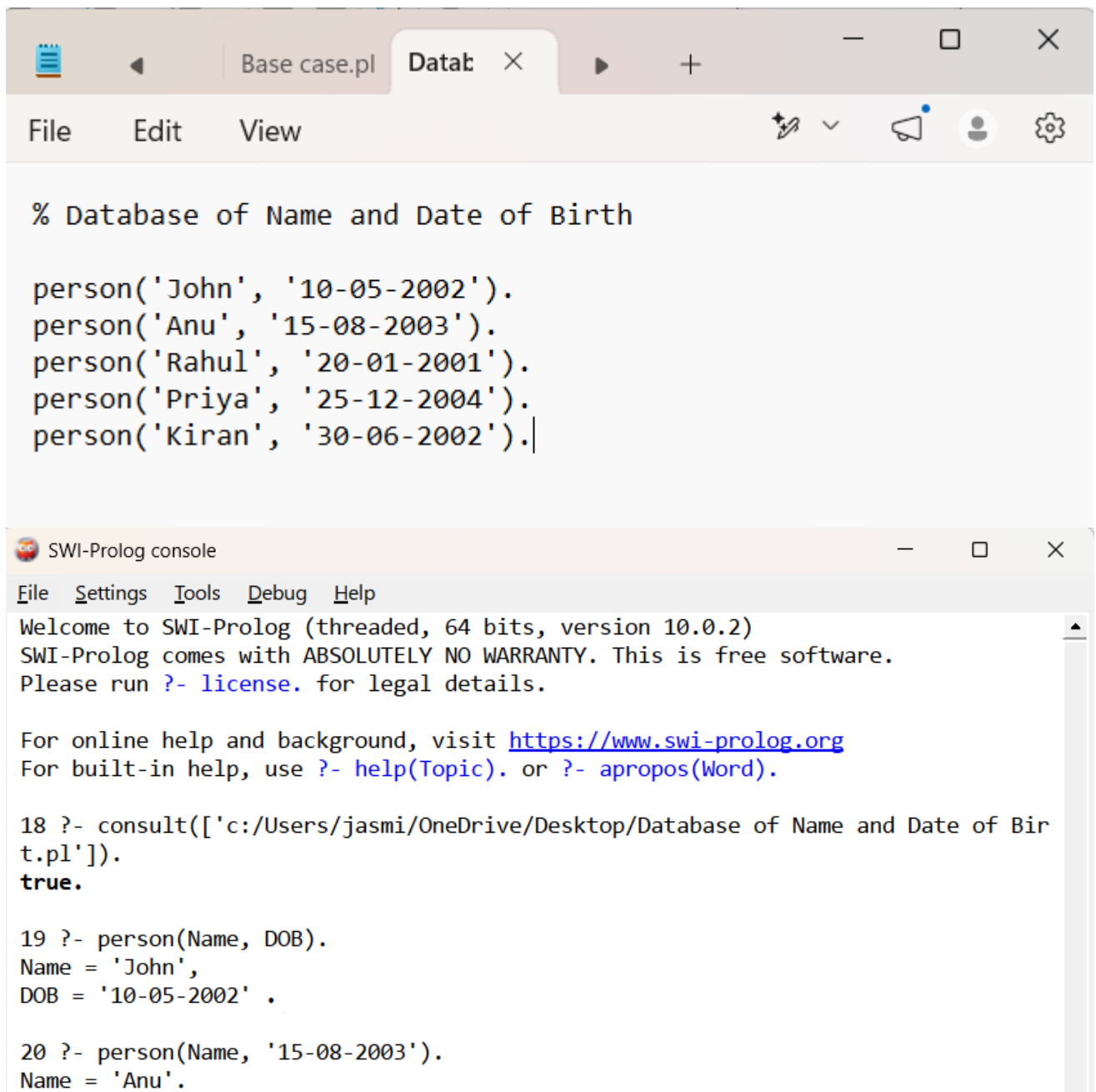
RESULT

The Prolog program was successfully implemented to calculate the sum of integers from 1 to N.

Program 18: Name and Date of Birth (DOB)

AIM

To create a database containing Name and Date of Birth (DOB) and display the stored records.



The screenshot displays two windows from the SWI-Prolog IDE. The top window, titled 'Base case.pl', contains a Prolog program that defines a database of names and dates of birth. The program consists of five facts: person('John', '10-05-2002'), person('Anu', '15-08-2003'), person('Rahul', '20-01-2001'), person('Priya', '25-12-2004'), and person('Kiran', '30-06-2002'). The bottom window, titled 'SWI-Prolog console', shows the execution of the program. It starts with a welcome message and then shows the execution of the query consult(['c:/Users/jasmi/OneDrive/Desktop/Database of Name and Date of Birth.pl']). The console output indicates that the consultation was successful (true). Subsequent queries for person(Name, DOB) are shown, with the first query returning Name = 'John' and DOB = '10-05-2002'.

```
% Database of Name and Date of Birth

person('John', '10-05-2002').
person('Anu', '15-08-2003').
person('Rahul', '20-01-2001').
person('Priya', '25-12-2004').
person('Kiran', '30-06-2002').
```

```
SWI-Prolog console

Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

18 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/Database of Name and Date of Birth.pl']).
true.

19 ?- person(Name, DOB).
Name = 'John',
DOB = '10-05-2002' .

20 ?- person(Name, '15-08-2003').
Name = 'Anu'.
```

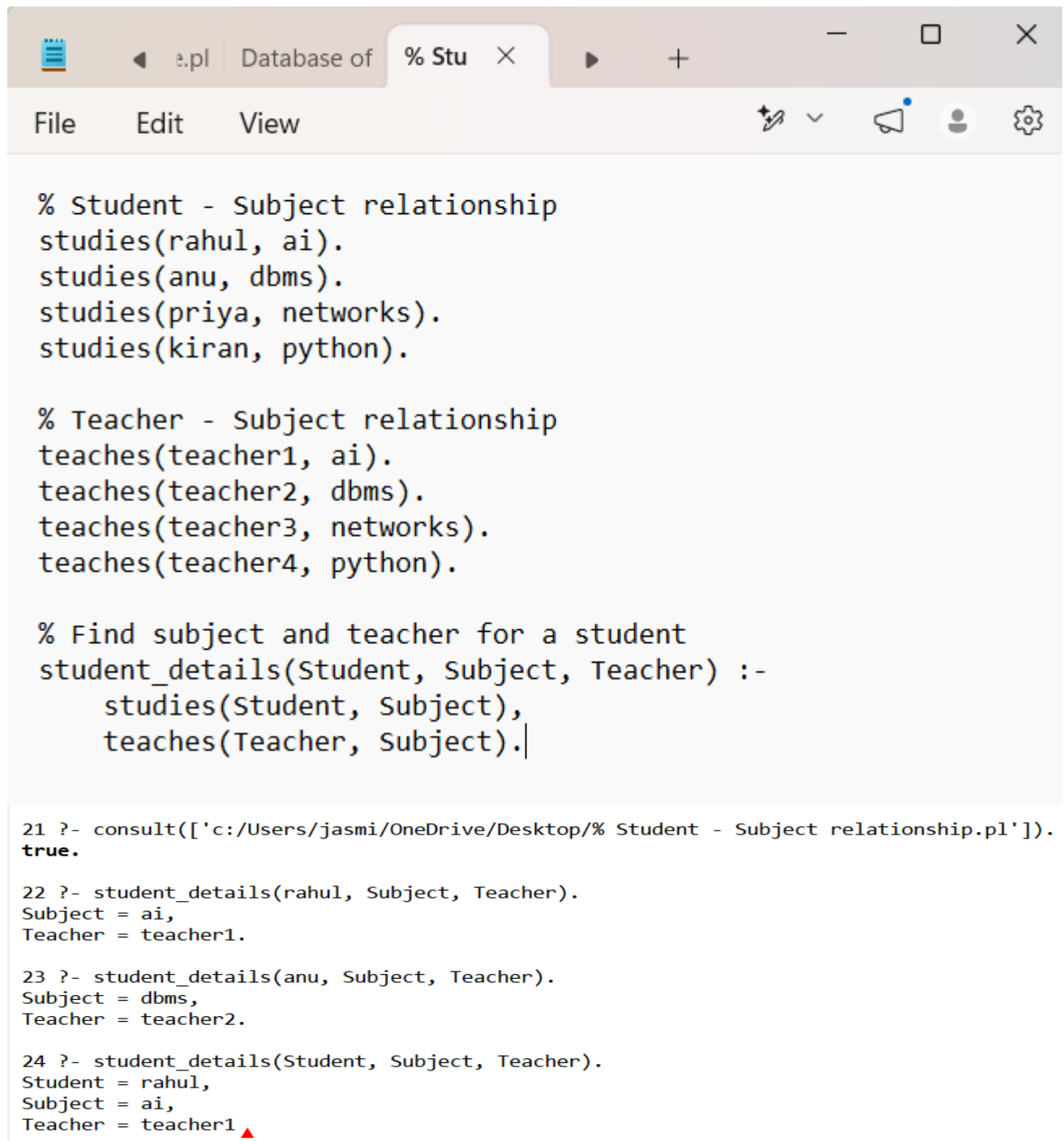
RESULT

The database was successfully created with Name and DOB fields, and the entered record was successfully stored and displayed.

Program 19: students, teachers, and subjects

AIM

To implement a Prolog program to represent the relationship between students, teachers, and subjects and display the subject and teacher corresponding to a given student.



```
% Student - Subject relationship
studies(rahul, ai).
studies(anu, dbms).
studies(priya, networks).
studies(kiran, python).

% Teacher - Subject relationship
teaches(teacher1, ai).
teaches(teacher2, dbms).
teaches(teacher3, networks).
teaches(teacher4, python).

% Find subject and teacher for a student
student_details(Student, Subject, Teacher) :-
    studies(Student, Subject),
    teaches(Teacher, Subject).

21 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/% Student - Subject relationship.pl']).
true.

22 ?- student_details(rahul, Subject, Teacher).
Subject = ai,
Teacher = teacher1.

23 ?- student_details(anu, Subject, Teacher).
Subject = dbms,
Teacher = teacher2.

24 ?- student_details(Student, Subject, Teacher).
Student = rahul,
Subject = ai,
Teacher = teacher1 ▲
```

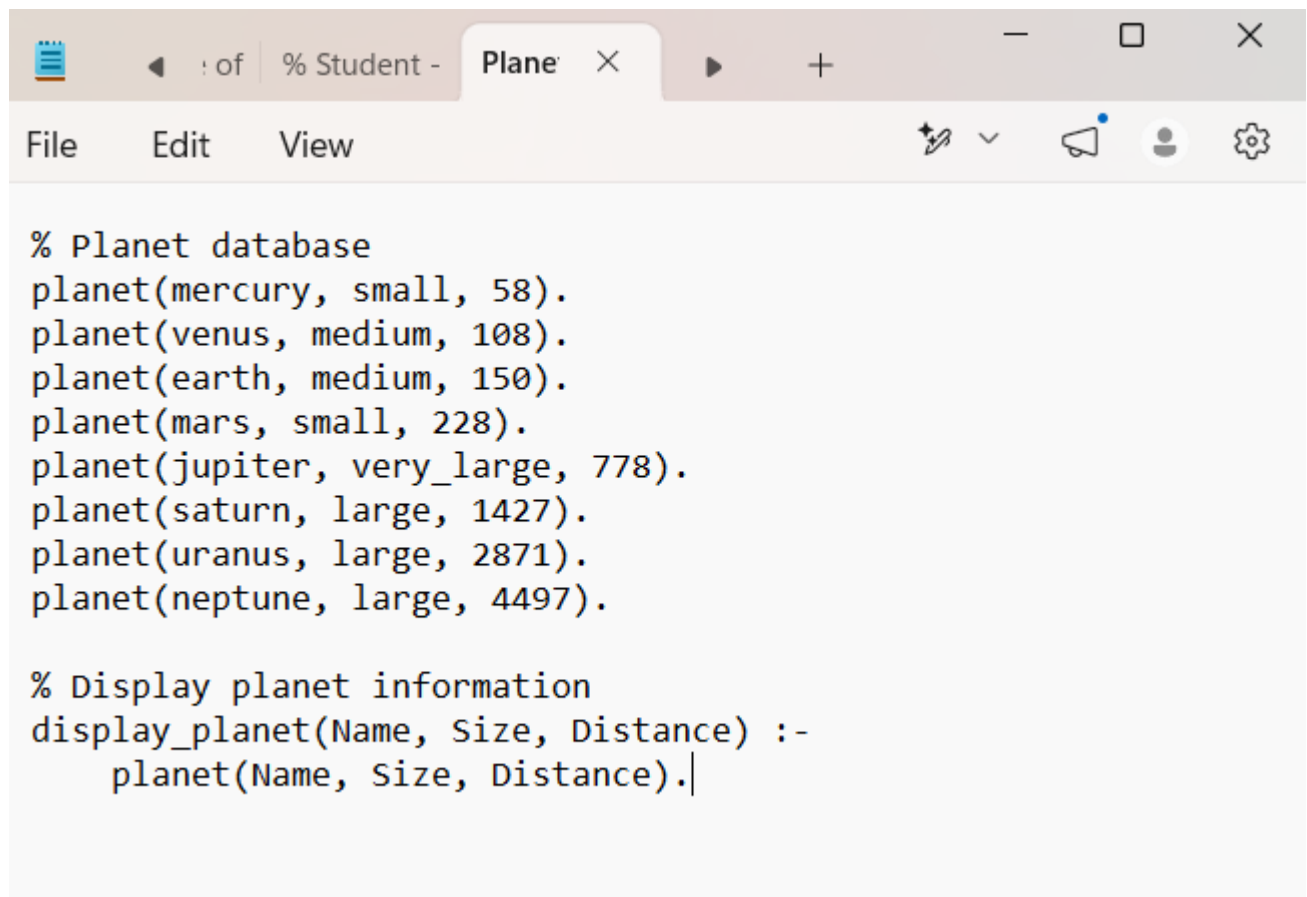
RESULT

The Student-Teacher-Subject relationship program was successfully implemented in Prolog.

Program 20: Planets database

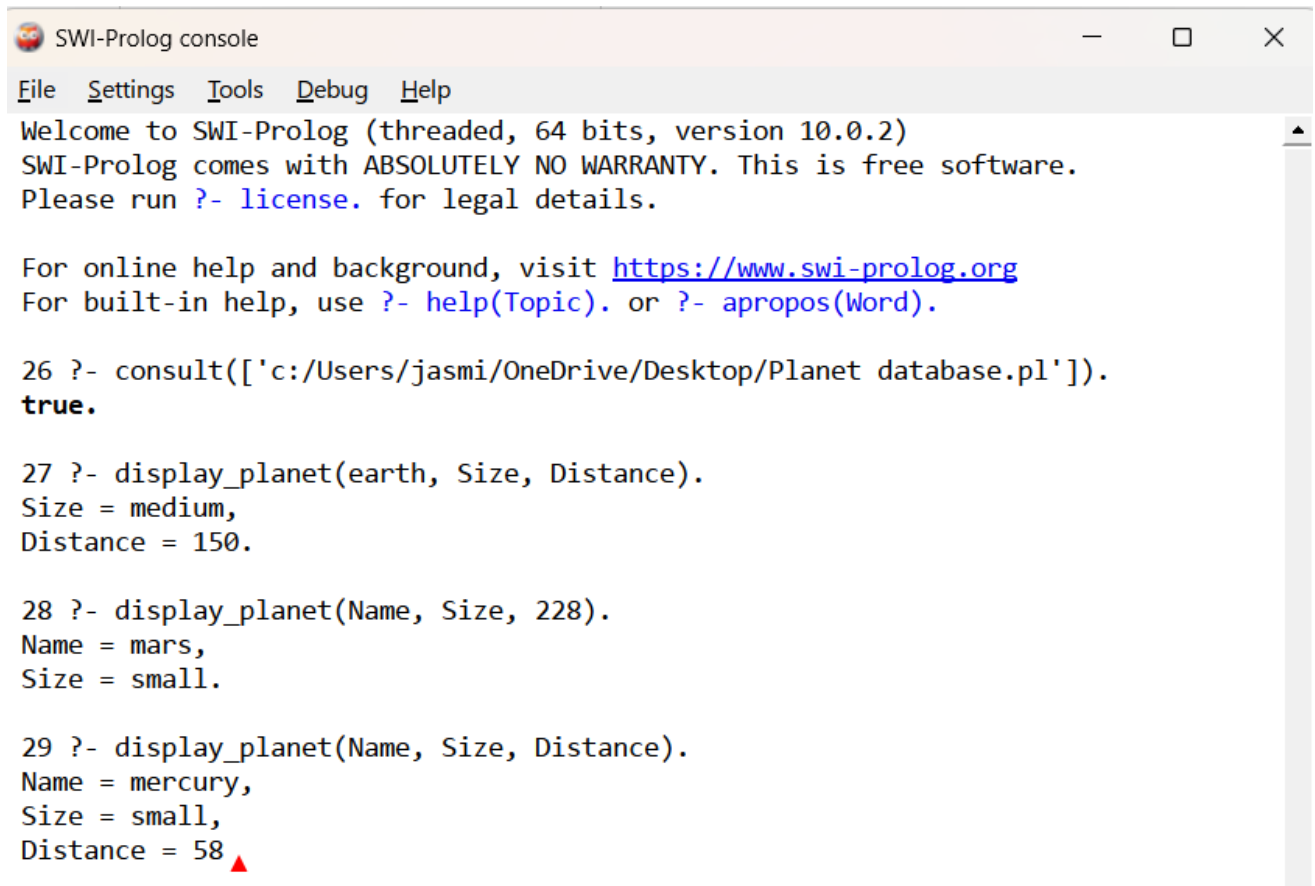
AIM

To implement a Prolog program to create a simple Planets database and store and display information such as planet name, size, and distance from the Sun.

A screenshot of a Prolog IDE window. The window has a title bar with a file icon, a back arrow, and the text ': of % Student - Plane'. Below the title bar is a menu bar with 'File', 'Edit', and 'View'. To the right of the menu bar are icons for a search, a dropdown arrow, a speaker, a user profile, and a settings gear. The main area of the window contains the following Prolog code:

```
% Planet database
planet(mercury, small, 58).
planet(venus, medium, 108).
planet(earth, medium, 150).
planet(mars, small, 228).
planet(jupiter, very_large, 778).
planet(saturn, large, 1427).
planet(uranus, large, 2871).
planet(neptune, large, 4497).

% Display planet information
display_planet(Name, Size, Distance) :-
    planet(Name, Size, Distance).
```


A screenshot of the SWI-Prolog console window. The window has a title bar with the SWI-Prolog logo and the text "SWI-Prolog console". Below the title bar is a menu bar with "File", "Settings", "Tools", "Debug", and "Help". The main area of the window contains the following text:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

26 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/Planet database.pl']).
true.

27 ?- display_planet(earth, Size, Distance).
Size = medium,
Distance = 150.

28 ?- display_planet(Name, Size, 228).
Name = mars,
Size = small.

29 ?- display_planet(Name, Size, Distance).
Name = mercury,
Size = small,
Distance = 58 ▲
```

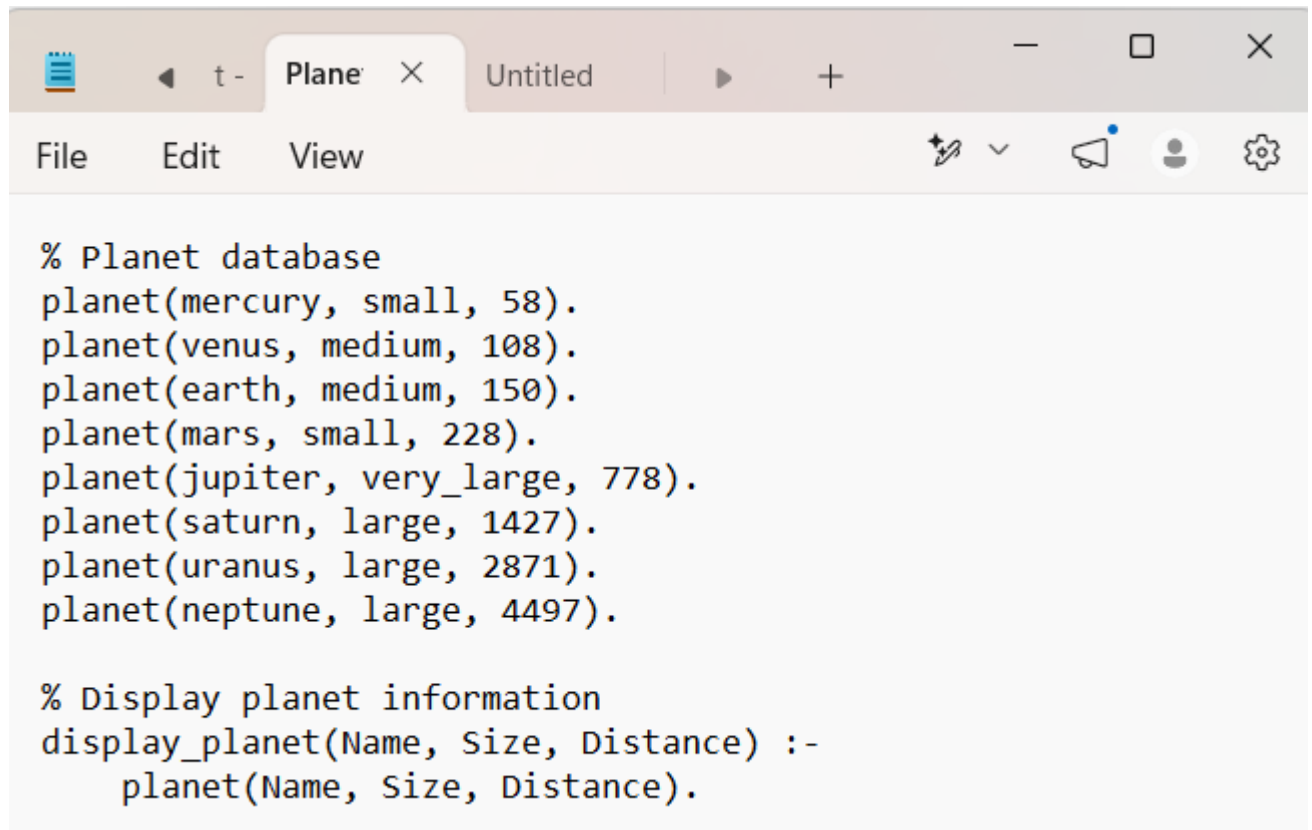
Result

Thus, the Prolog program was successfully implemented to create a Planets database and store and retrieve information about the planet name, size, and distance from the Sun.

Program 21: Tower of Hanoi problem

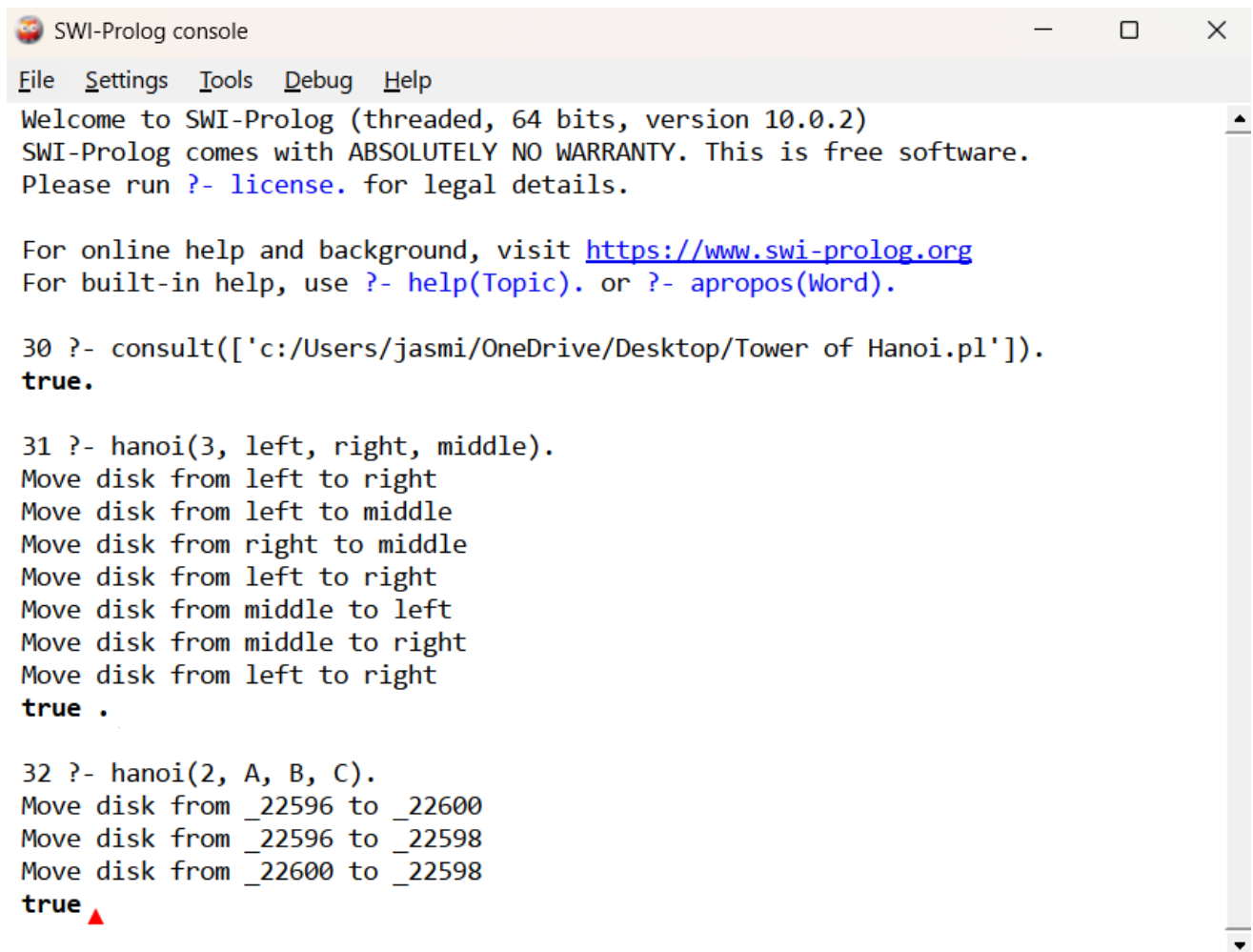
AIM

To implement the Tower of Hanoi problem using recursion in Python and display the sequence of moves required to transfer all disks from the source rod to the destination rod.



```
% Planet database
planet(mercury, small, 58).
planet(venus, medium, 108).
planet(earth, medium, 150).
planet(mars, small, 228).
planet(jupiter, very_large, 778).
planet(saturn, large, 1427).
planet(uranus, large, 2871).
planet(neptune, large, 4497).

% Display planet information
display_planet(Name, Size, Distance) :-
    planet(Name, Size, Distance).
```

A screenshot of a SWI-Prolog console window. The window has a title bar with a red icon and the text "SWI-Prolog console". Below the title bar is a menu bar with "File", "Settings", "Tools", "Debug", and "Help". The main text area contains the following output:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

30 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/Tower of Hanoi.pl']).
true.

31 ?- hanoi(3, left, right, middle).
Move disk from left to right
Move disk from left to middle
Move disk from right to middle
Move disk from left to right
Move disk from middle to left
Move disk from middle to right
Move disk from left to right
true .

32 ?- hanoi(2, A, B, C).
Move disk from _22596 to _22600
Move disk from _22596 to _22598
Move disk from _22600 to _22598
true ▲
```

RESULT

Thus, the Tower of Hanoi problem was successfully implemented in Prolog, and the required sequence of disk movements was obtained.

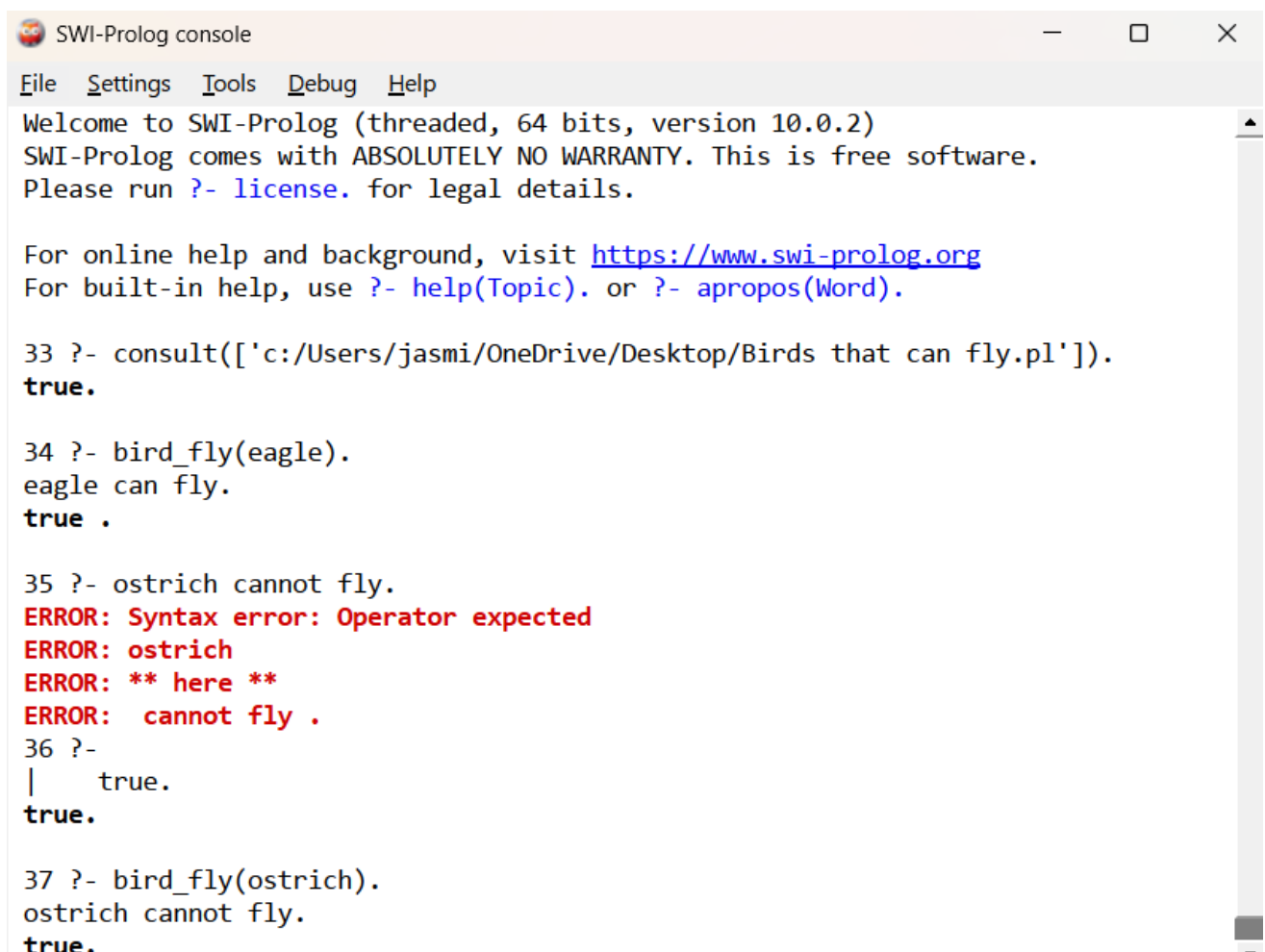
Program 22:

Aim

To implement a Prolog program to determine whether a particular bird can fly or not and incorporate the required queries.

Algorithm – Bird Can Fly or Not

1. Start the program.
2. Define the list of birds that can fly.
3. Define the list of birds that cannot fly.
4. Define a rule `bird_fly(Bird)` to check the given bird.
5. If the bird is present in `can_fly`, display "Bird can fly."
6. If the bird is present in `cannot_fly`, display "Bird cannot fly."
7. Give the required query with a particular bird name.
8. Display the result.
9. Stop the program.



```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

33 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/Birds that can fly.pl']).
true.

34 ?- bird_fly(eagle).
eagle can fly.
true .

35 ?- ostrich cannot fly.
ERROR: Syntax error: Operator expected
ERROR: ostrich
ERROR: ** here **
ERROR: cannot fly .
36 ?-
| true.
true.

37 ?- bird_fly(ostrich).
ostrich cannot fly.
true.
```

```
% Birds that can fly
can_fly(eagle).
can_fly(parrot).
can_fly(pigeon).
can_fly(crow).
can_fly(sparrow).

% Birds that cannot fly
cannot_fly(penguin).
cannot_fly(ostrich).
cannot_fly(emu).

% Rule to check whether a bird can fly
bird_fly(Bird) :-
    can_fly(Bird),
    write(Bird),
    write(' can fly.').

bird_fly(Bird) :-
    cannot_fly(Bird),
    write(Bird),
    write(' cannot fly.').
```

Result

Thus, the Prolog program was successfully implemented to determine and print whether a particular bird can fly or not using facts and rules.

Program 23:

Aim

To implement a Family Tree using Prolog and represent relationships such as parent, father, mother, brother, sister, and grandparent.

Algorithm

1. Start the program.
2. Define facts for parent, male, and female.
3. Define the rule for father.
4. Define the rule for mother.
5. Define the rule for brother and sister.
6. Define the rule for grandparent.
7. Give the required query.
8. Display the relationship.
9. Stop.



File Edit View

% Facts

```
male(john).
male(robert).
male(david).
male(james).
```

```
female(mary).
female(susan).
female(lisa).
female(anna).
```

```
parent(john, robert).
parent(mary, robert).
```

```
parent(john, susan).
parent(mary, susan).
```

```
parent(robert, david).
parent(lisa, david).
```

```
parent(robert, anna).
parent(lisa, anna).
```

% Rules

```
father(X, Y) :-
    male(X),
    parent(X, Y).
```

```
mother(X, Y) :-
    female(X),
    parent(X, Y).
```

```
brother(X, Y) :-
    male(X),
    parent(P, X),
    parent(P, Y),
    X \= Y.
```

```
sister(X, Y) :-
    female(X),
    parent(P, X),
    parent(P, Y),
    X \= Y.
```

```
grandparent(X, Y) :-
    parent(X, P),
    parent(P, Y).
```

A screenshot of the SWI-Prolog console window. The window has a title bar with the text "SWI-Prolog console" and standard Windows window controls (minimize, maximize, close). Below the title bar is a menu bar with "File", "Settings", "Tools", "Debug", and "Help". The main text area contains the following text:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

40 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/Family tree.pl']).
true.

41 ?- father(X, robert).
X = john .

42 ?- mother(X, robert).
X = mary .

43 ?- sister(X, robert).
X = susan .
```

Result

Thus, the Family Tree was successfully implemented in Prolog, and the required family relationships were obtained using suitable queries.

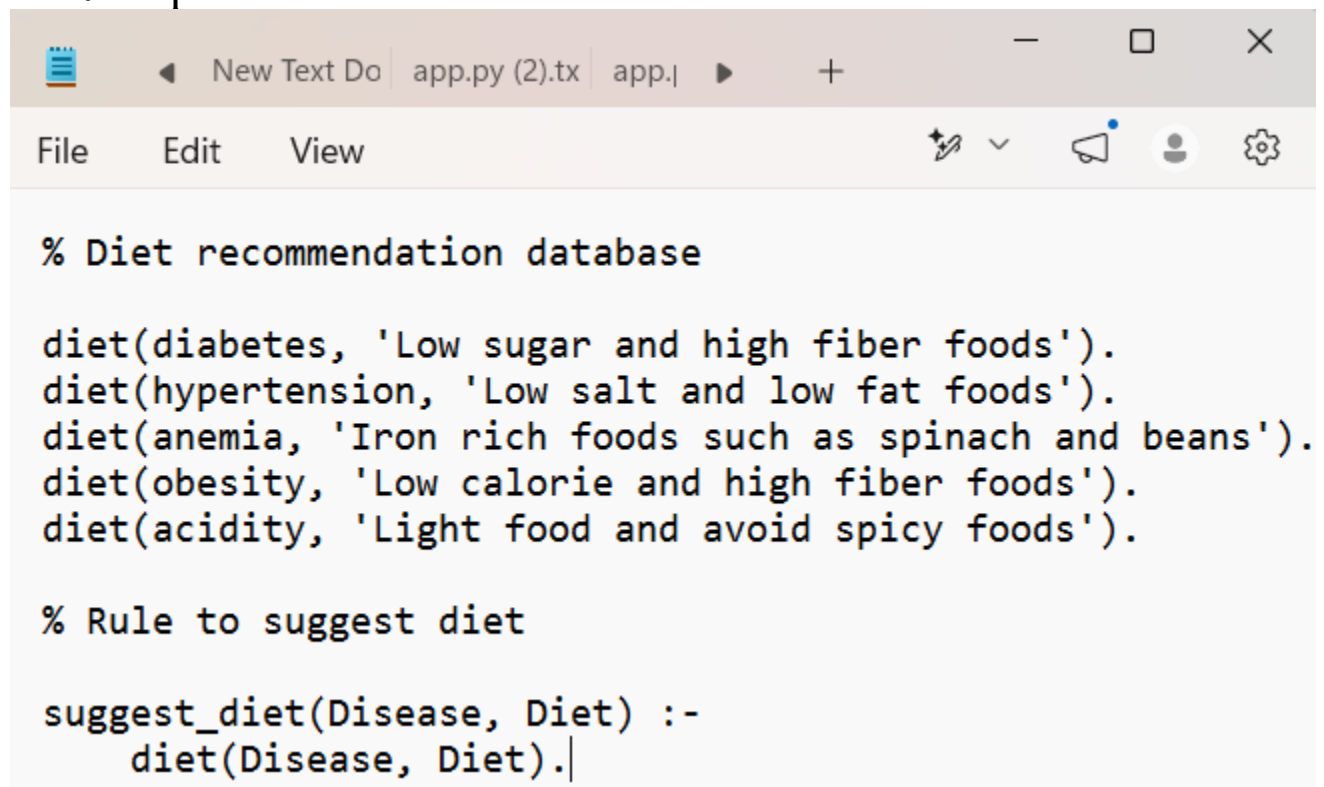
Program 24:

Aim

To implement a Prolog program to suggest a suitable diet based on a given disease.

Algorithm

1. Start the program.
2. Define diseases and their recommended foods.
3. Define rules to suggest a diet for each disease.
4. Accept the disease as input through a query.
5. Match the given disease with the corresponding diet.
6. Display the recommended foods.
7. Stop.

A screenshot of a text editor window titled 'New Text Do' with tabs for 'app.py (2).tx' and 'app.j'. The editor shows Prolog code for a diet recommendation system. The code includes a database of diseases and their recommended diets, a rule to suggest a diet, and a query to find the diet for a specific disease.

```
% Diet recommendation database

diet(diabetes, 'Low sugar and high fiber foods').
diet(hypertension, 'Low salt and low fat foods').
diet(anemia, 'Iron rich foods such as spinach and beans').
diet(obesity, 'Low calorie and high fiber foods').
diet(acidity, 'Light food and avoid spicy foods').

% Rule to suggest diet

suggest_diet(Disease, Diet) :-
    diet(Disease, Diet).|
```

A screenshot of the SWI-Prolog console window. The window has a title bar with the text "SWI-Prolog console" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with "File", "Settings", "Tools", "Debug", and "Help". The main text area contains the following text:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

45 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/Diet recommendation database.pl'
]).
true.

46 ?- suggest_diet(diabetes, Diet).
Diet = 'Low sugar and high fiber foods'.

47 ?- suggest_diet(anemia, Diet).
Diet = 'Iron rich foods such as spinach and beans'.
```

Result

Thus, the Prolog program was successfully implemented to suggest a diet based on a given disease using facts and rules.

Program 25:

Aim

To implement the Monkey Banana Problem using Prolog, where a monkey must move a box to reach the bananas hanging from the ceiling.

Algorithm

1. Start the program.
2. Define the initial position of the monkey, box, and bananas.
3. Check whether the monkey can reach the box.
4. If necessary, move the box below the bananas.
5. Move the monkey onto the box.
6. Check whether the monkey is below the bananas.
7. If the monkey is below the bananas and standing on the box, allow it to grasp the bananas.
8. Display "Monkey got the bananas."
9. Stop.

 app.py (2) | index.html | app.py | edge_detect | jasmitl • | fami
File Edit View

```
% Monkey Banana Problem  
  
% Initial state:  
% Monkey is at door, box is at window, bananas are at middle.  
  
% Move monkey from one place to another  
move_monkey(At, To, State, NewState) :-  
    State = state(At, Box, Bananas, Floor),  
    NewState = state(To, Box, Bananas, Floor).  
  
% Push the box  
push_box(At, To, State, NewState) :-  
    State = state(At, At, Bananas, Floor),  
    NewState = state(To, To, Bananas, Floor).  
  
% Climb onto the box  
climb_box(State, NewState) :-  
    State = state(Pos, Pos, Bananas, Floor),  
    NewState = state(Pos, Pos, Bananas, on_box).  
  
% Grasp the bananas  
grasp(State) :-  
    State = state(Pos, Pos, Pos, on_box),  
    write('Monkey got the bananas!'),  
    nl.
```

A screenshot of the SWI-Prolog console window. The window has a title bar with a red icon and the text "SWI-Prolog console". Below the title bar is a menu bar with "File", "Settings", "Tools", "Debug", and "Help". The main text area contains the following output:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

50 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/Monkey Banana Problem.pl']).
Warning: c:/users/jasmi/onedrive/desktop/monkey\_banana\_problem.pl:17:
Warning: Singleton variables: [Floor]
true.

51 ?- grasp(state(middle, middle, middle, on_box)).
Monkey got the bananas!
true.

52 ?- grasp(state(door, window, middle, on_floor)).
false.

53 ?- ▲
```

Result

Thus, the Monkey Banana Problem was successfully implemented in Prolog, and the program determines whether the monkey can obtain the bananas based on its position, the box position, and the banana position.

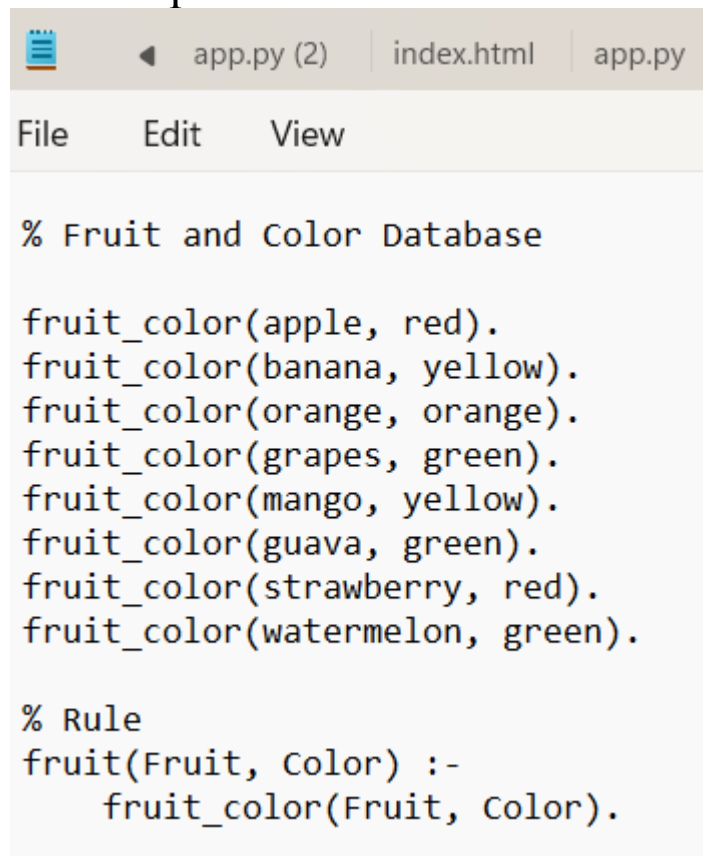
Program 26:

Aim

To implement a Prolog program to store fruits and their colors and display the results using backtracking.

Algorithm

1. Start the program.
2. Define facts containing the fruit and its color.
3. Create a rule to retrieve the fruit and its color.
4. Give a query with variables.
5. Prolog displays the first matching fruit and color.
6. Press ; to perform **backtracking** and get the next matching result.
7. Continue until all matching facts are displayed.
8. Stop.

A screenshot of a text editor window showing a Prolog program. The window has a title bar with three tabs: 'app.py (2)', 'index.html', and 'app.py'. The menu bar includes 'File', 'Edit', and 'View'. The code is written in Prolog and defines a database of fruit colors and a rule to query them. The code is as follows:

```
% Fruit and Color Database

fruit_color(apple, red).
fruit_color(banana, yellow).
fruit_color(orange, orange).
fruit_color(grapes, green).
fruit_color(mango, yellow).
fruit_color(guava, green).
fruit_color(strawberry, red).
fruit_color(watermelon, green).

% Rule
fruit(Fruit, Color) :-
    fruit_color(Fruit, Color).
```

A screenshot of the SWI-Prolog console window. The window has a title bar with the text "SWI-Prolog console" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with "File", "Settings", "Tools", "Debug", and "Help". The main text area contains the following output:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

58 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/Fruit and Color Database.pl']).
true.

59 ?- fruit(Fruit, Color).
Fruit = apple,
Color = red ▲
```

Result

Thus, the Prolog program was successfully implemented to represent fruits and their colors using facts and retrieve multiple solutions through backtracking.

Program 27:

AIM

To write and implement a Prolog program for Best-First Search algorithm to find a path from a starting node to a goal node using heuristic values.

ALGORITHM

1. Start with the initial node.
2. Add the initial node to the OPEN list.
3. Assign heuristic values $H(n)$ to the nodes.
4. Select the node having the lowest heuristic value from the OPEN list.
5. Check whether the selected node is the goal node.
6. If it is the goal, stop and display the path.
7. Otherwise, expand the selected node and generate its successors.
8. Add the unvisited successors to the OPEN list.
9. Sort the OPEN list according to heuristic values.
10. Repeat steps 4–9 until the goal is reached.



app.py

edge_detect

jasmitl



family.pl

File Edit View

```
% Best First Search in Prolog

% Graph representation
edge(a, b).
edge(a, c).
edge(b, d).
edge(b, e).
edge(c, f).
edge(c, g).
edge(d, h).
edge(e, h).
edge(f, h).
edge(g, h).

% Heuristic values
h(a, 7).
h(b, 6).
h(c, 4).
h(d, 3).
h(e, 2).
h(f, 3).
h(g, 1).
h(h, 0).

% Best First Search
best_first(Start, Goal, Path) :-
    best_first_search([[Start]], Goal, RevPath),
    reverse(RevPath, Path).

best_first_search([[Goal|Rest]|_], Goal, [Goal|Rest]).

best_first_search([[Node|Path]|Open], Goal, Result) :-
    findall(
        [Next|Node|Path],
        (edge(Node, Next),
         \+ member(Next, [Node|Path])),
        Children
    ),
    append(Open, Children, NewOpen),
    sort_by_heuristic(NewOpen, SortedOpen),
    best_first_search(SortedOpen, Goal, Result).

% Sort paths according to heuristic value
sort_by_heuristic(Paths, Sorted) :-
    map_list_to_pairs(path_heuristic, Paths, Pairs),
    keysort(Pairs, SortedPairs),
    pairs_values(SortedPairs, Sorted).

path_heuristic([Node|_], H) :-
    h(Node, H).
```

A screenshot of the SWI-Prolog console window. The window has a title bar with the text "SWI-Prolog console" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with "File", "Settings", "Tools", "Debug", and "Help". The main text area contains the following text:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

62 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/Best First Search in Prolog.pl']
).
true.

63 ?- best_first(a, h, Path).
Path = [a, c, g, h] ▲
```

RESULT

The Best-First Search algorithm was successfully implemented in Prolog, and the optimal heuristic-based path from the starting node to the goal node was obtained.

Program 28:

AIM

To write and implement a Prolog program for medical diagnosis that identifies a possible disease based on the symptoms provided by the user.

ALGORITHM

1. Start the program.
2. Define diseases and their corresponding symptoms as facts/rules.
3. Accept the symptoms from the user.
4. Compare the given symptoms with the symptoms associated with each disease.
5. If the symptoms match the conditions of a disease, identify the possible disease.
6. Display the diagnosis.
7. Stop the program.



app.py

edge_detect

jasmitl



family.pl

File Edit View

```
% Medical Diagnosis System
```

```
disease(flu) :-  
    symptom(fever),  
    symptom(cough),  
    symptom(headache).
```

```
disease(cold) :-  
    symptom(cough),  
    symptom(sneezing),  
    symptom(runny_nose).
```

```
disease(malaria) :-  
    symptom(fever),  
    symptom(chills),  
    symptom(sweating).
```

```
disease(typhoid) :-  
    symptom(fever),  
    symptom(headache),  
    symptom(abdominal_pain).
```

```
diagnose(Disease) :-  
    disease(Disease).
```

```
% Symptoms  
symptom(fever).  
symptom(cough).  
symptom(headache).|
```

A screenshot of the SWI-Prolog console window. The window has a title bar with the text "SWI-Prolog console" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with "File", "Settings", "Tools", "Debug", and "Help". The main text area contains the following text: "Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)", "SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.", "Please run ?- license. for legal details.", "For online help and background, visit <https://www.swi-prolog.org>", "For built-in help, use ?- help(Topic). or ?- apropos(Word).", "67 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/% Medical Diagnosis System.pl'])", ".
true.", "68 ?- diagnose(Disease).", "Disease = flu ▲".

```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

67 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/% Medical Diagnosis System.pl'])
.
true.

68 ?- diagnose(Disease).
Disease = flu ▲
```

Result:

The medical diagnosis system was successfully implemented in Prolog and a possible disease was identified based on the given symptoms.

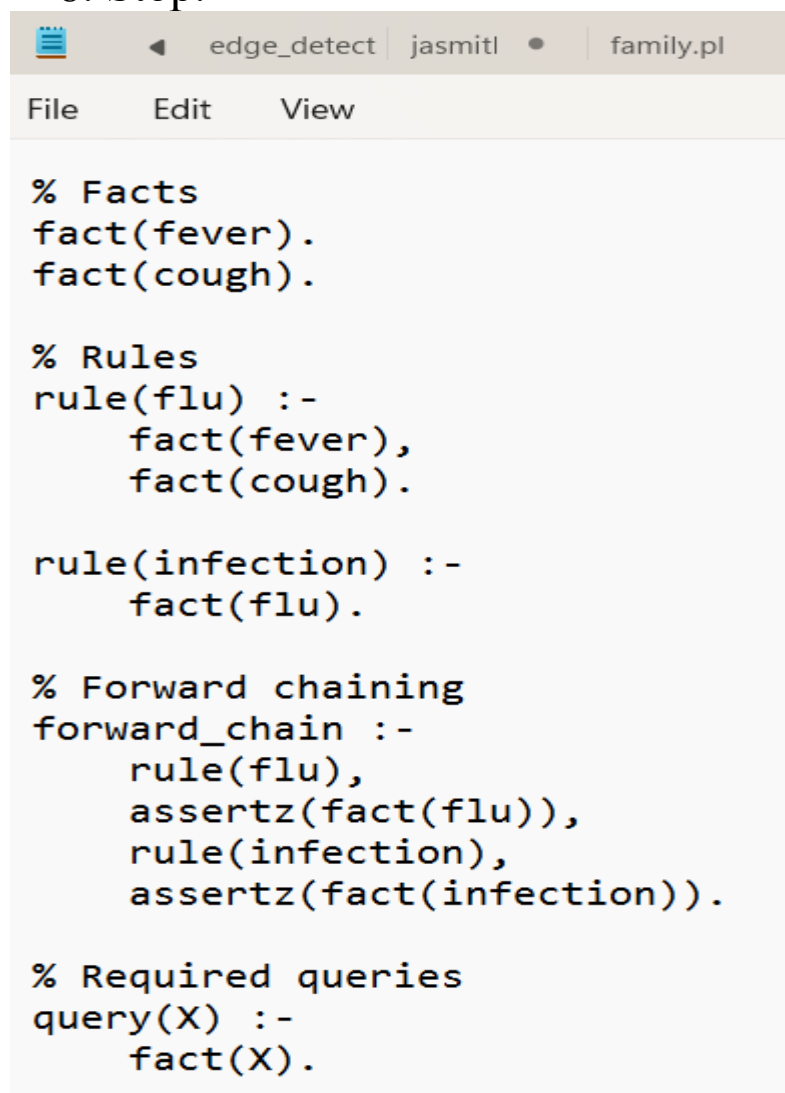
Program 29:

AIM

To write and implement a Prolog program for Forward Chaining and incorporate the required queries to derive new facts from existing facts and rules.

ALGORITHM

1. Start with a set of known facts.
2. Define rules using the known facts.
3. Check whether the conditions of any rule are satisfied.
4. If satisfied, derive a new fact.
5. Add the new fact to the knowledge base.
6. Repeat the process until no new facts can be derived.
7. Execute the required queries to verify the derived facts.
8. Stop.

A screenshot of a text editor window showing a Prolog program. The window has a title bar with a file icon, a back arrow, and the text 'edge_detect | jasmitl • family.pl'. Below the title bar is a menu bar with 'File', 'Edit', and 'View'. The main text area contains the following Prolog code:

```
% Facts
fact(fever).
fact(cough).

% Rules
rule(flu) :-
    fact(fever),
    fact(cough).

rule(infection) :-
    fact(flu).

% Forward chaining
forward_chain :-
    rule(flu),
    assertz(fact(flu)),
    rule(infection),
    assertz(fact(infection)).

% Required queries
query(X) :-
    fact(X).
```



```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

70 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/% Facts.pl']).
true.

71 ?- fact(fever).
true.

72 ?-
|    fact(cough).
true.
```

RESULT

Thus, the Forward Chaining algorithm was successfully implemented in Prolog, and the required queries were executed to derive new facts from the given knowledge base.

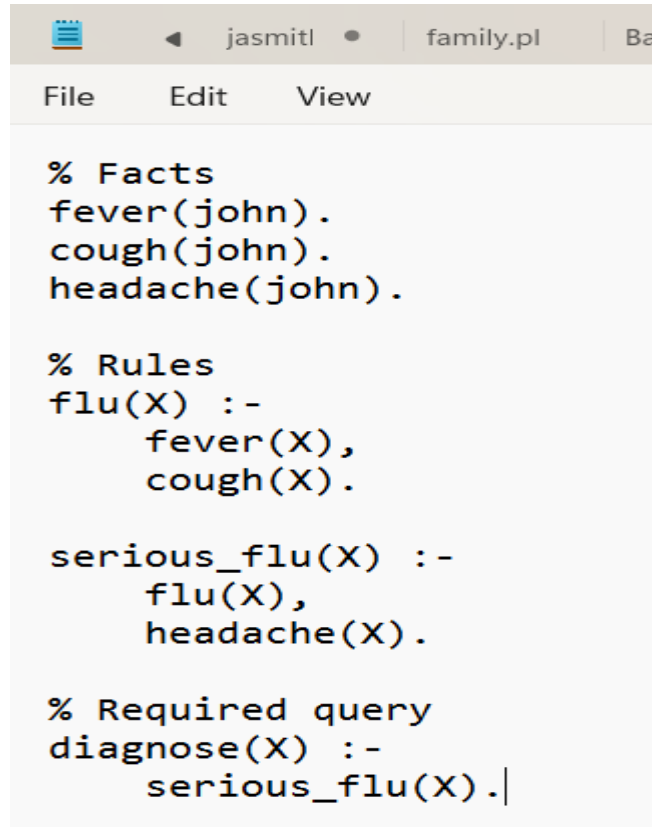
Program 30:

AIM

To write and implement a Prolog program for Backward Chaining and execute the required queries to prove a goal using the available facts and rules.

ALGORITHM

1. Start with a goal to be proved.
2. Search for a rule whose conclusion matches the goal.
3. Treat the conditions of that rule as sub-goals.
4. Check whether each sub-goal can be proved using facts or other rules.
5. Continue recursively until the required facts are found.
6. If all sub-goals are satisfied, the original goal is proved.
7. Execute the required queries and display the result.
8. Stop.

A screenshot of a text editor window showing a Prolog program. The window has a title bar with a file icon, a back arrow, the text 'jasmitl', a bullet point, the filename 'family.pl', and a partial window title 'Ba'. Below the title bar is a menu bar with 'File', 'Edit', and 'View'. The main text area contains the following Prolog code:

```
% Facts
fever(john).
cough(john).
headache(john).

% Rules
flu(X) :-
    fever(X),
    cough(X).

serious_flu(X) :-
    flu(X),
    headache(X).

% Required query
diagnose(X) :-
    serious_flu(X).|
```


A screenshot of the SWI-Prolog console window. The window has a title bar with the text "SWI-Prolog console" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with "File", "Settings", "Tools", "Debug", and "Help". The main text area contains the following text: "Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)", "SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.", "Please run ?- license. for legal details.", "For online help and background, visit <https://www.swi-prolog.org>", "For built-in help, use ?- help(Topic). or ?- apropos(Word).", "75 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/Facts.pl']).", "true.", "76 ?- flu(john).", "true.", "77 ?- flu(mary).", "false.". The text is displayed in a monospaced font, with the prompt "?-" in blue and the results "true." and "false." in black. The file path in the third query is highlighted in blue. The window has a vertical scrollbar on the right side.

```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

75 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/Facts.pl']).
true.

76 ?- flu(john).
true.

77 ?- flu(mary).
false.
```

RESULT

Thus, the Backward Chaining algorithm was successfully implemented in Prolog, and the required queries were executed successfully to prove the given goals from the available facts and rules.

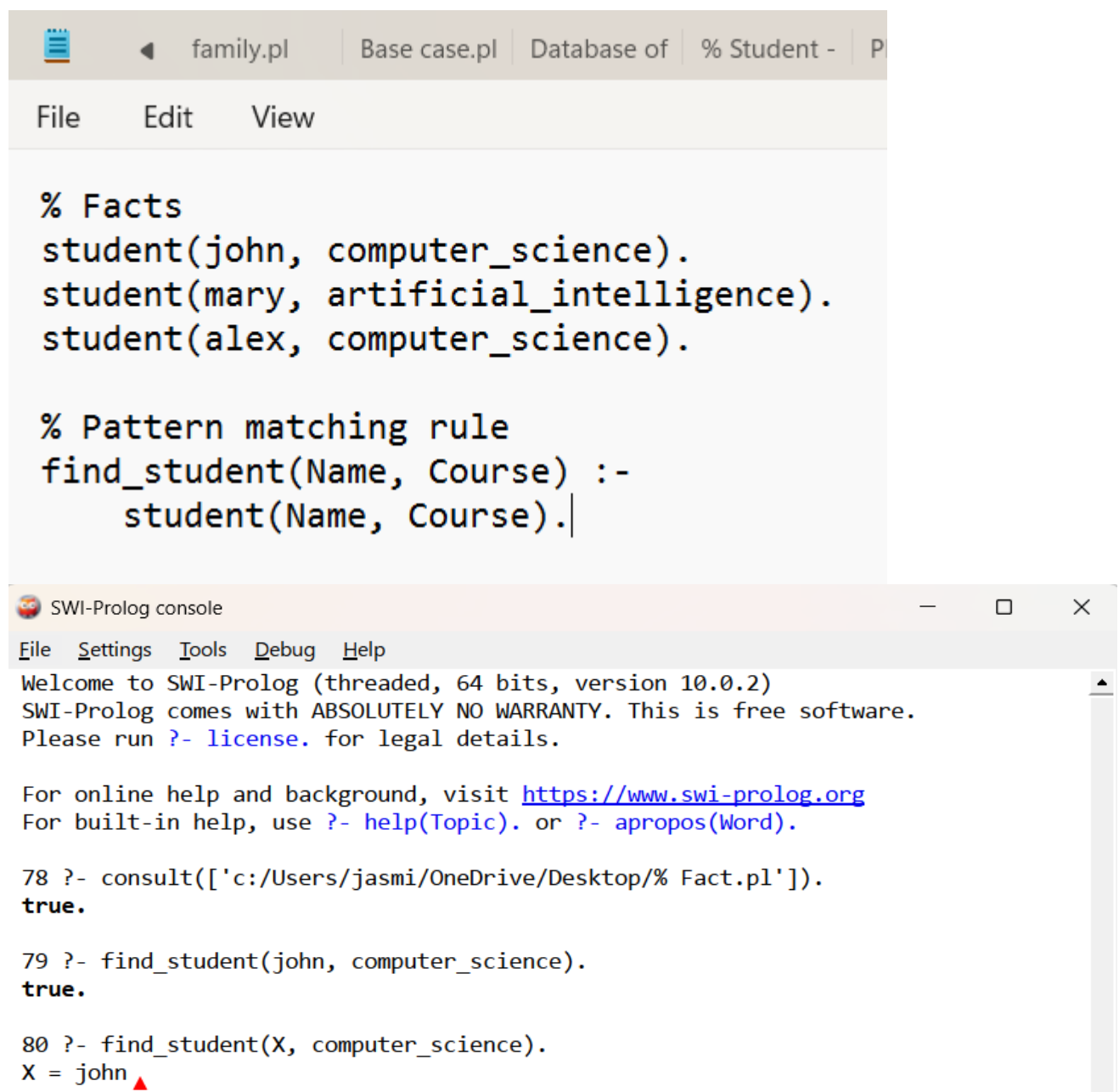
Program 31:

AIM

To write and implement a Prolog program for Pattern Matching and execute the required queries to match given patterns with facts.

ALGORITHM

1. Start the program.
2. Define facts containing different patterns.
3. Define a rule to match the given pattern with the facts.
4. Use Prolog variables to find matching values.
5. Execute the required queries.
6. Display the matched result.
7. Stop.



```
family.pl | Base case.pl | Database of | % Student - | P
File Edit View

% Facts
student(john, computer_science).
student(mary, artificial_intelligence).
student(alex, computer_science).

% Pattern matching rule
find_student(Name, Course) :-
    student(Name, Course).

SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

78 ?- consult(['c:/Users/jasmi/OneDrive/Desktop/% Fact.pl']).
true.

79 ?- find_student(john, computer_science).
true.

80 ?- find_student(X, computer_science).
X = john
```

RESULT

Thus, the Pattern Matching program was successfully implemented in Prolog, and the required queries were executed to obtain the matching values.

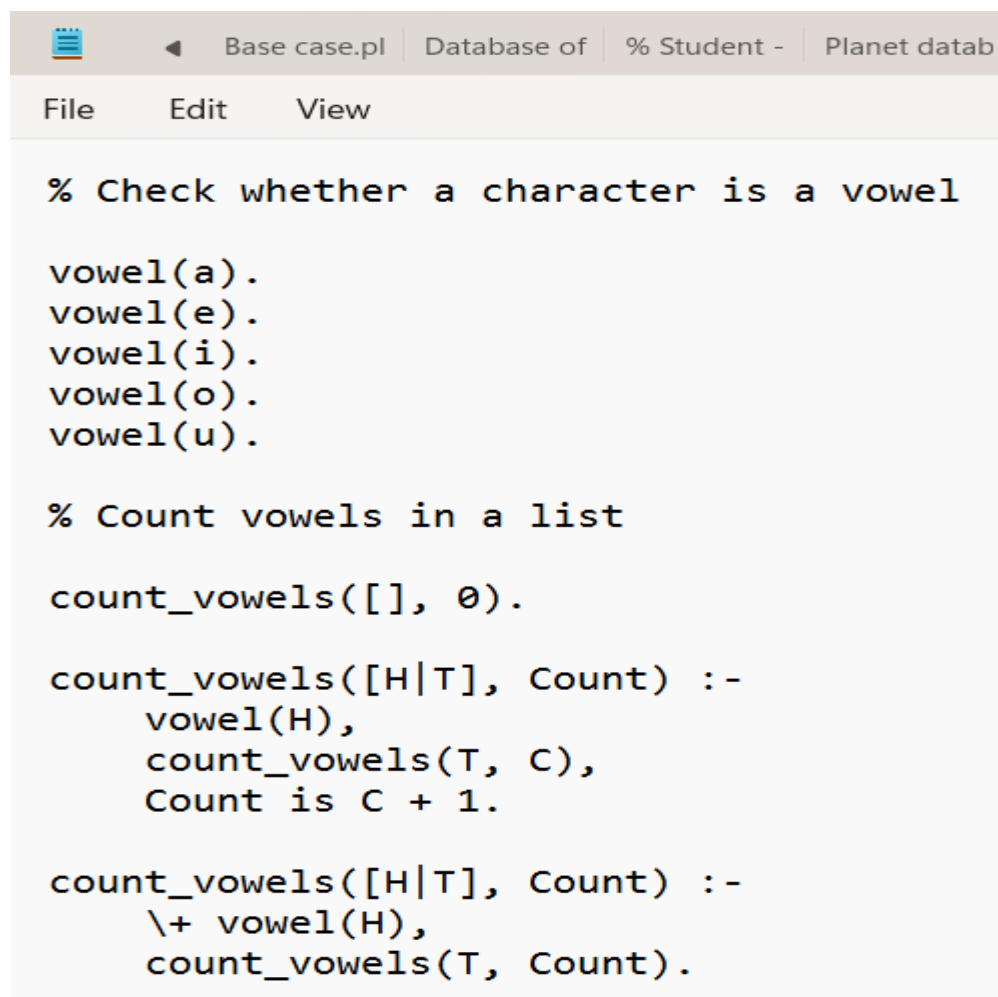
Program 32:

AIM

To write and implement a Prolog program to find the number of vowels in a given string.

ALGORITHM

1. Start the program.
2. Read a string as input.
3. Check each character of the string.
4. If the character is a vowel (a, e, i, o, u), increment the count.
5. Continue until all characters are checked.
6. Display the total number of vowels.
7. Stop.

A screenshot of a Prolog IDE window. The title bar shows a file icon and several tabs: 'Base case.pl', 'Database of', '% Student -', and 'Planet datab'. The menu bar includes 'File', 'Edit', and 'View'. The main text area contains the following Prolog code:

```
% Check whether a character is a vowel

vowel(a).
vowel(e).
vowel(i).
vowel(o).
vowel(u).

% Count vowels in a list

count_vowels([], 0).

count_vowels([H|T], Count) :-
    vowel(H),
    count_vowels(T, C),
    Count is C + 1.

count_vowels([H|T], Count) :-
    \+ vowel(H),
    count_vowels(T, Count).
```

A screenshot of the SWI-Prolog console window. The window has a title bar with the text "SWI-Prolog console" and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with "File", "Settings", "Tools", "Debug", and "Help". The main text area contains the following content:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

84 ?- consult(['c:/Users/jasmi/OneDrive/Pictures/Documents/Check whether a character is a vo.pl']).
true.

85 ?- count_vowels([a,e,i,o,u], N).
N = 5 .

86 ?- count_vowels([h,e,l,l,o], N).
N = 2 ▲
```

RESULT

Thus, the Prolog program to find the number of vowels in a given list was successfully implemented, and the number of vowels was obtained correctly.